

Random Multi-Type Spanning Forests for Synchronization on Sparse Graphs*

Hugo Jaquard, Pierre-Olivier Amblard, Simon Barthelmé, Nicolas Tremblay

CNRS, Univ. Grenoble Alpes, Grenoble INP, GIPSA-lab, France

Abstract. Random diffusions are a popular tool in Monte-Carlo estimations, with well established algorithms such as Walk-on-Spheres (WoS) going back several decades. In this work, we introduce diffusion estimators for the problems of angular synchronization and smoothing on graphs, in the presence of a rotation associated to each edge. Unlike classical WoS algorithms that are point-wise estimators, our diffusion estimators allow for *global* estimations by propagating along the branches of random spanning subgraphs called *multi-type spanning forests*. Building upon efficient samplers based on variants of Wilson’s algorithm, we show that our estimators outperform standard numerical-linear-algebra solvers in challenging instances, depending on the topology and density of the graph.

Key words. RandNLA, graphs, random spanning forests, connection, graph signal processing, synchronization

AMS subject classifications. 68W20, 68Q87, 60G35, 62M99

Introduction. Data processing over graphs is of interest in many fields of research, such as discrete geometry, signal processing or graph machine learning. A common setting is that of nodes supporting some kind of data, typically numerical values, whereas the edges describe the topology of the underlying space. In a number of situations, the edges can carry more precise geometric information regarding *how* two data points defined on adjacent nodes should be compared, and thus processed. In geometrical parlance, this information is known as a *connection* (an idea first formalized in [13]).

We focus here on a connection describing rotations of angle $\theta_{i,j}$ associated to each edge (i, j) , a setting which has found a growing number of applications over the last decade, including *angular synchronization* [75, 84], signal processing over directed graphs [26, 87], discrete geometry processing [71], or even direction of arrival estimation [55, 65]. Solving these problems requires computing the solution of large numerical-linear-algebra (NLA) problems, whose formulation relies on the *connection Laplacian* L_θ (the formal definition of L_θ is postponed to Section 1.2), a linear operator encoding both the topology of the graph *and* the rotations $\theta_{i,j}$.

Computing an exact solution to many of these problems is prohibitive beyond moderately large graphs ($n \simeq 10^4$)¹, so that approximations via iterative Krylov-subspace-based algorithms (such as the popular Conjugate Gradient algorithm) are typically used instead [67, 68]. The main drawback of these methods is their spectrum-dependent convergence rate [43], that often requires hard-to-find, good-quality preconditioners to ensure fast convergence.

Randomized numerical linear algebra (RandNLA) is a successful and modern alternative [21, 52]: Monte-Carlo estimators allow flexible schemes of computation (*e.g.*, parallelized or distributed) and can exhibit both advantageous complexities and state-of-the-art practical performances, *in spite of* their slow convergence rates in $O\left(\frac{\sigma}{\sqrt{m}}\right)$ (with σ^2 the variance of the estimator and m the number of Monte-Carlo samples). Methods specialized in graphs, based

*This work is an extension of the conference proceedings [36].

¹For instance, Tikhonov smoothing requires solving a linear system, with generic time complexity in $\mathcal{O}(n^3)$.

on random walks and decompositions of the graph, have also appeared (e.g. [45, 61, 77]).

In this paper, we leverage novel connection-aware random decompositions of the graph, and propose RandNLA estimators for two connection-Laplacian-based problems:

- A graph Tikhonov smoothing problem in the presence of a connection.
- The angular synchronization problem on graphs.

Tikhonov smoothing. Connection-aware graph Tikhonov smoothing amounts to solving:

$$(0.1) \quad f_* = \operatorname{argmin}_{f \in \mathbf{C}^n} q \|f - g\|_2^2 + \frac{1}{2} \sum_{i,j} |f(j) - e^{t\theta_{i,j}} f(i)|^2,$$

where $g \in \mathbf{C}^n$ is a complex graph signal one wishes to smooth (e.g., denoise), $q \in \mathbf{R}_+^*$ is an *a priori* known regularization parameter, and the sum is over a subset of ordered pairs of indices (i, j) describing the edges of the graph. This problem appears in different contexts, such as vector field extension in discrete geometry processing [71], or directed graph signal processing [26]. Here, the regularization term (the sum) penalizes functions that are not *locally coherent* with respect to the connection. The solution to Eq. (0.1) can be expressed using the connection Laplacian L_θ : $f_* = q(L_\theta + qI)^{-1}g$. This inverse operation has a cubic complexity in n and is prohibitive for large graphs, requiring the use of efficient estimators. Connection-aware Tikhonov smoothing also appears as an intermediate step in angular synchronization (the second problem we consider) as well as in iterative solvers for yet other problems².

Angular Synchronization. The objective is to recover a set of n unknown angles $\omega = (\omega)_i \subseteq [0, 2\pi)^n$ from measured pairwise offset measurements $\{\theta_{i,j}\}_{i,j}$ [75]:

$$(0.2) \quad \theta_{i,j} = \omega_j - \omega_i + \varepsilon_{i,j} \mod 2\pi,$$

where $\varepsilon_{i,j}$ represents some unknown degradation of the measurement. This task appears in many structured signal processing problems, where it is often a key component in state-of-the-art recovery methods, such as perceived luminance reconstruction [83], ptychography [24], ranking [16], clock synchronization [28] or phase reconstruction [1], and also appears in the statistical physics literature (e.g. [14]).

In practice, we may only observe a subset of all such measurements $\theta_{i,j}$, and the problem is naturally formulated on a graph $\mathcal{G}$ with n nodes whose set of edges $\mathcal{E}$ is indexed by the number of measurements, and where edge (i, j) (resp. (j, i)) carries the offset $\theta_{i,j}$ (resp. $(\theta_{j,i} = -\theta_{i,j})$). If noiseless measurements $\theta_{i,j} = \omega_j - \omega_i$ are available, exact recovery can be trivially performed up to a global phase shift, by *propagating* values according to the offset measurements along a spanning tree of $\mathcal{G}$ (a procedure explained in Figures 0.1a and 0.1b).

The problem becomes more involved when considering imperfect measurements $\theta_{i,j}$, with non-zero noise $\varepsilon_{i,j}$. In general, *exact* angular synchronization can no longer be performed in this noisy regime, as long as even one *incoherent* cycle is present in the graph (see Figure 0.1c).

²This is for instance the case for the so-called “edge-lasso” over graphs [72] (which only appears in the literature on graphs *without* a connection, but is relevant on graphs with a connection as well), which corresponds to an l_1 -regularization problem over graph signals. The solution to this problem can for instance be obtained from iteratively-reweighted least-squares (see, e.g., [61]), or from an ADMM strategy (e.g., [27]), or other proximal-operator-based algorithms [59]; all of these strategies may involve solving graph Tikhonov regularization problems.

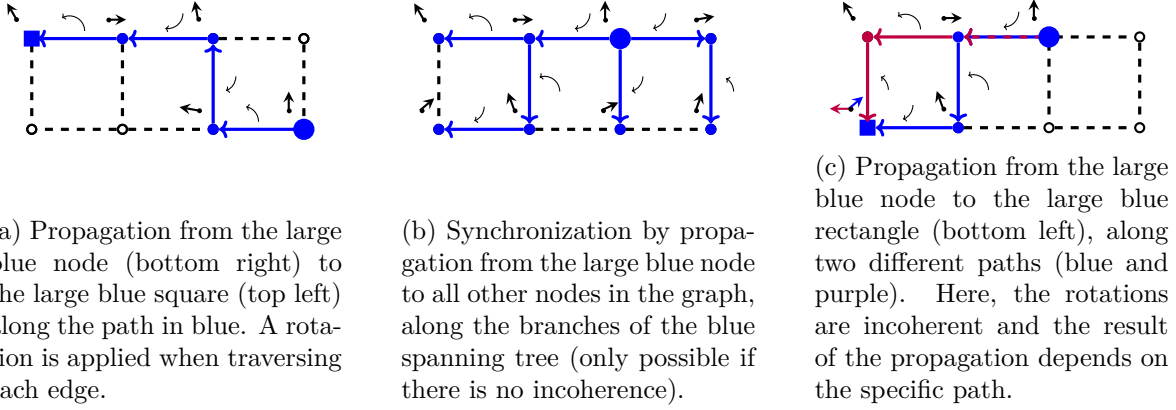


Figure 0.1: Propagations on the 4×2 grid graph, along different paths. The rotation angles $\theta_{i,j}$'s associated to each edge are represented as circular rotating arrows (only drawn along the paths we consider). Propagations always start from the large blue node, and propagated angles are represented as straight arrows (top left of each node). Left 0.1a: propagation along an arbitrary path. Center 0.1b: exact synchronization achieved by propagation along a spanning tree, in the absence of noise. Right 0.1c: synchronization impossible due to noise and incoherence along cycles.

A common workaround, first introduced in [83], consists in quantifying the incoherence of an angular assignment $s \in [0, 2\pi)^n$ as

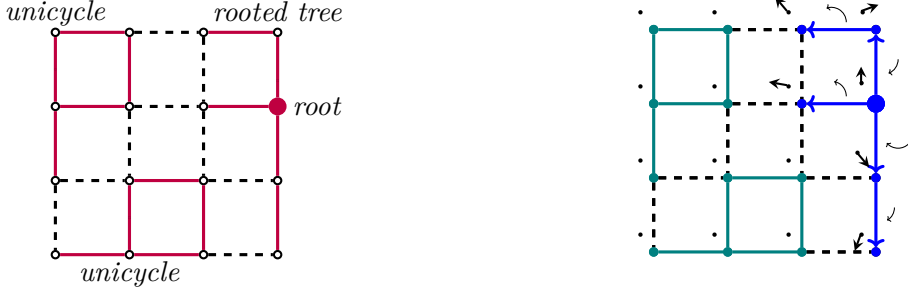
$$(0.3) \quad \mathcal{I}(s) = \sum_{\{i,j\} \in \mathcal{E}} (2 - 2 \cos((s_j - s_i) - \theta_{i,j})),$$

before minimizing this incoherence over all possible assignments. While this problem is non-convex and NP-hard in general [10, 86], different techniques allowing to recover a solution have been proposed [10, 32, 60, 75, 84], be it approximately via relaxations or for specific noise regimes or topologies. Most of these methods rely on the *connection Laplacian* L_θ .

Our contributions. We propose novel RandNLA estimators for the smoothing and angular synchronization problems. We rely on propagations along branches of *Multi-Type Spanning Forests* (MTSF): spanning subsets of both edges and nodes of a graph, whose connected components are either rooted trees or unicycles (connected subsets of edges containing *exactly one* cycle). Our main theorem concerns our estimator for the graph Tikhonov smoothing problem, and can be roughly described as follows (see Theorem 2.4 for a formal statement).

Theorem 0.1. *For MTSFs sampled from the correct distribution (the one of Eq. (2.4)), propagating within each tree the value of its root to all of its other nodes yields an unbiased estimation of the solution of the connection-aware Tikhonov smoothing problem of Eq. (0.1).*

See Figure 0.2 for an illustration. We include a teaser runtime comparison with a standard deterministic solver in Figure 0.3, which shows that our estimators are *less sensitive* to the density of the graph, and can provide significant speed-ups for equivalent precision.



(a) A MTSF in purple, with two unicycles (top left, bottom) and one tree (right). The large purple node is the root of the tree.

(b) Estimation. Tree: the value of the root (blue circle) is propagated to the rest of the tree via the blue arrows. Unicycles: the estimation is 0 (arrowless dots at the top left of the nodes).

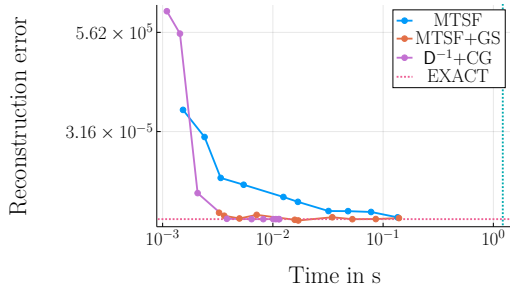
Figure 0.2: Theorem 0.1 illustrated on the 4×4 grid graph. Left: a MTSF. Right: estimation by propagations along the branches of the trees of the MTSF. The estimation in $\mathbf{C}$ is represented using angled arrows (top left of the nodes).

In practice, we obtain a *fast* and *scalable* algorithm. MTSF-sampling is achieved via a Wilson-like random-walk-based algorithm with runtime *linear* in the number of edges of the graph. On the theoretical side, our arguments rely on the theory of *Determinantal Point Processes* (DPPs) [34, 51], and generalize the combinatorial analyses of [41, 61].

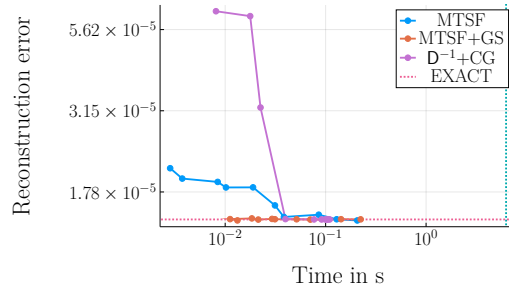
A preliminary version of this work already appeared in [36], where we presented our smoothing estimator along with two variance-reduction techniques, and an application to a ranking problem for a simple synthetic data model. We improve and extend this work in a number of different manners, some of which we list below.

Theoretical results. In addition to Theorem 2.4, we derive a connection-aware Feynman-Kac formula (Proposition 2.2), resulting in a local (node-wise) random-walk based estimator for the smoothing problem, similar to walk-on-spheres algorithms [56, 69]. In comparison, our MTSF-based estimators allow for *global* estimation on all the nodes at once. We further relate these two estimators, which may pave the way to generalizations (see Section 16 of the Supplementary Material). We also improve one of our variance-reduction techniques to better handle heterogeneous degree distributions, with significant improvements.

Methodological and experimental contributions. We compare our estimators with standard Krylov-subspaces methods on synthetic data, for both the angular synchronization and smoothing problems. For the angular synchronization problem, we leverage our smoothing estimator as an iterative step in existing approaches [10, 75, 84]. We analyze and contrast the behavior of these methods on different graph topologies which, up to our knowledge, has never been studied in the literature before. Our results show that MTSF-based estimations can outperform standard deterministic methods whenever the graph is not *very sparse* (the gain getting bigger as the density increases). See Figure 0.3. The code used for these



(a) DC-SBM with mean degree $\bar{d} \simeq 30$.



(b) DC-SBM with mean degree $\bar{d} \simeq 420$.

Figure 0.3: Runtime-precision comparisons for the graph Tikhonov smoothing problem, comparing two MTSF-based estimators (MTSF and MTSF+GS) against a diagonally-preconditioned conjugate-gradient-descent ($D^{-1}+CG$) on degree-corrected stochastic block model graphs (DC-SBM, see Section 3 for a definition) with 10000 nodes. Results are averaged over 10 graphs generated from DC-SBM models with two different parametrizations, resulting in different average degrees $\bar{d}$: on the left, a DC-SBM with average degree $\bar{d} \simeq 30$ and, on the right, a DC-SBM with average degree $\bar{d} \simeq 420$. x -axis: average runtime. y -axis: average reconstruction error. Each data-point corresponds to a number $m \in \{1, 2, 3, 5, 8, 13, 22, 36, 60, 100\}$ of MTSFs (or of CG iterations) used in the estimation. The vertical (resp. horizontal) line is the computation-time (resp. error) of an exact Cholesky solver (see Section 3 for details).

experiments is publicly available³.

Related Work. Our main result can be understood as a specialization of the approach in [19, 20], based on DPPs, a class of distributions exhibiting negatively-correlated samples [34, 44], and resulting in unbiased estimators for least-squares problems. {However, and in contrast to generic DPPs that are in general expensive to sample from, the specific structure of MTSFs allows for very efficient sampling algorithms and, in turn, practical graph RandNLA algorithms that can compete with deterministic state-of-the-art solvers.

There exists similar applications of random spanning forests distributions in RandNLA for problems on connection-free graphs [3, 4], such as the trace estimation of (regularized) inverse Laplacians [6] or, the Tikhonov regularization and interpolation of graphs signals [61]. Multi-Type Spanning Forests have also been used to build spectral sparsifiers for the (regularized) connection Laplacian in [22], resulting in randomized preconditioners. Our Feynman-Kac formula is inspired from a similar result in [40] for continuous-time random walks, but is specially tailored to the graph Tikhonov smoothing problem; discrete-time walks also exhibit intriguing links with propagations on MTSFs (see Section 16 of the Supplementary Material).

Organisation of the Paper. In Section 1, we introduce preliminary background on graphs and connections used in the rest of the paper. In Section 2, we present our Feynman-Kac formula (Section 2.1) and our MTSF-based estimators (Section 2.2), analyze an efficient sampling algorithm for MTSFs (Section 2.3), and describe our variance-reduction techniques (Sec-

³<https://gricad-gitlab.univ-grenoble-alpes.fr/gaia/synchromtsf>

tion 2.4). We analyze the numerical behavior of our estimators in Section 3, and compare them with standard deterministic algorithms (Sections 3.1 and 3.2). In Section 4, we propose an iterative scheme leveraging our smoothing estimators to solve the angular synchronization problem (Section 4.1), illustrate its application to a denoising problem inspired from *cryogenic electron microscopy* (Section 4.2), and compare this scheme with standard deterministic methods (Section 4.3). Proofs are deferred to the Supplementary Material.

1. Background. A graph $\mathcal{G}$ is defined as a set on nodes $\mathcal{V}$ interconnected by a set of edges $\mathcal{E}$. The edges of $\mathcal{E}$ are *non-oriented* edges. However, for the purpose of this paper, we will also need to consider their *oriented* counterparts, denoted by $\vec{\mathcal{E}}$. The size of $\vec{\mathcal{E}}$ is twice the size of $\mathcal{E}$: each edge $e \in \mathcal{E}$ is associated to two oriented edges in $\vec{\mathcal{E}}$. Choosing arbitrarily an orientation for e and writing s_e and t_e its *source* and *target*, we have $e = (s_e, t_e) \in \vec{\mathcal{E}}$, and its reversely-oriented edge $e^* = (t_e, s_e) \in \vec{\mathcal{E}}$. We state our results for *unweighted* graphs, but all our results generalize to the *weighted* case, as detailed in the Supplementary Material.

1.1. Combinatorial Laplacian. An elementary instance of graph-supported data is that of real values attached to the nodes of the graph, usually formalized as a vector $f \in \mathbf{R}^{\mathcal{V}}$, and called a *graph signal* [74]. The regularity of such a signal can be quantified using the *combinatorial Laplacian* $\mathbf{L} : \mathbf{R}^{\mathcal{V}} \rightarrow \mathbf{R}^{\mathcal{V}}$, a symmetric semi-definite positive operator, conveniently expressed as $\mathbf{L} = \mathbf{D} - \mathbf{A}$, where $\mathbf{D}$ is the diagonal degree matrix ($D_{i,i} = d_i$ the degree of node i) and $\mathbf{A}$ the adjacency matrix of the graph (with $A_{(i,j)} = 1$ if $(i,j) \in \vec{\mathcal{E}}$ and 0 otherwise) [15]. The quadratic form associated to $\mathbf{L}$ acts as:

$$(1.1) \quad \langle f, \mathbf{L}f \rangle = \frac{1}{2} \sum_{e \in \vec{\mathcal{E}}} |f(t_e) - f(s_e)|^2,$$

which associates a high value to functions with important local variations over the edges of the graph, and serves as a basis for the notion of *frequency* for graph signals [74, 79]. These operators are also related to combinatorial properties of the graph $\mathcal{G}$ [8], for instance:

(P1) $\ker \mathbf{L}$ is always non-empty, with dimension the number of connected components of $\mathcal{G}$.

(P2) $\mathbf{D}^{-1}\mathbf{A}$ is the matrix of the natural *random walk* on $\mathcal{G}$, which transitions from a node to one of its neighbors with uniform probability at each step, with $\left((\mathbf{D}^{-1}\mathbf{A})^l\right)_{i,j}$ the probability that a path of length l starting at i ends up in node j .

1.2. Connection Laplacian: Definition and Basic Properties. *Unitary connections* are a practical way to add additional information to the graph's structure⁴. We describe in the following how to capture additional rotations associated to edges in the form of a Laplacian-like operator. The main idea is to represent rotations as *unitary complex numbers*, which will naturally lead to consider *complex-valued* functions $f \in \mathbf{C}^{\mathcal{V}}$ when studying variational properties on $\mathcal{G}$. The entries of f should be understood as belonging to different copies $\mathbf{C}_v$ of the complex plane $\mathbf{C}$ associated to each node $v \in \mathcal{V}$, see Figure 1.1⁵.

⁴By describing explicit geometrical transformations between the signal values of the nodes of $\mathcal{G}$.

⁵This association of a copy of $\mathbf{C}$ is analogous to a fiber bundle over a manifold (e.g. its tangent bundle), and is known as a *complex line bundle* over $\mathcal{G}$.

170 We will associate to each directed edge in $\vec{\mathcal{E}}$ a map representing the transformation along
 171 that edge, which is known as a *connection* [41]. A *unitary connection* Ψ on a graph $\mathcal{G}$ is
 172 a collection of unitary linear maps $(\psi_e)_{e \in \vec{\mathcal{E}}}$, with each map $\psi_e : \mathbf{C}_{s_e} \rightarrow \mathbf{C}_{t_e}$ acting as
 173 multiplication by a unitary complex number $e^{i\theta_e}$, so that $\psi_e(z) = e^{i\theta_e} \cdot z$, with i the complex
 174 imaginary unit (we sometimes abuse notation and write $\psi_e = e^{i\theta_e}$ when there is no risk
 175 of confusion). In addition, we require that $\psi_{e^*} = \psi_e^*$ (i.e. $\psi_{(t_e, s_e)} = \psi_{(s_e, t_e)}^*$), so that the
 176 transformation associated to an edge traversed in one direction or the other differs only by
 177 conjugation. This notion is illustrated in Figure 1.1.

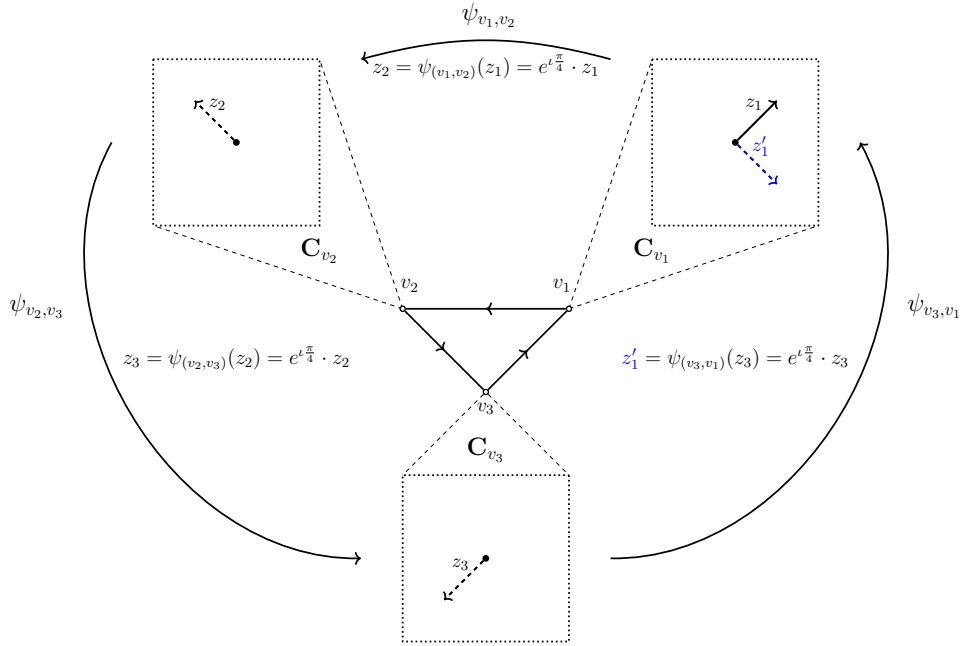


Figure 1.1: A connection Ψ on the triangle graph K_3 (in the center). The connection maps ψ_e are associated with angles $\theta_{(v_1, v_2)} = \theta_{(v_2, v_3)} = \theta_{(v_3, v_1)} = \frac{\pi}{4}$, and we represent their action on some $z_1 \in \mathbf{C}_{v_1}$. A cyclic path $C = ((v_1, v_2), (v_2, v_3), (v_3, v_1))$ is depicted using directed arrows, we denote by $\psi_C = \psi_{(v_3, v_1)} \circ \psi_{(v_2, v_3)} \circ \psi_{(v_1, v_2)}$ the composition of the connection maps along this cycle (a rotation of $\theta_C = \frac{3\pi}{4}$). Top right: $z_1 \in \mathbf{C}_{v_1}$ (bold arrow) and $z_1' = \psi_C(z_1) \in \mathbf{C}_{v_1}$ (blue dotted arrow). Top left, bottom: $z_2 = \psi_{(v_1, v_2)}(z_1) \in \mathbf{C}_{v_2}$ and $z_3 = \psi_{(v_2, v_3)}(z_2) \in \mathbf{C}_{v_3}$.

178 **Remark 1.1.** In applications, such as angular synchronization, the θ_e 's often represent a
 179 priori known rotations, and are frequently modulated by a scale parameter $\gamma \in \mathbf{R}_+$, resulting
 180 in connections such that $\psi_e(z) = e^{i\gamma\theta_e} z$. While we will forego this additional parameter in our
 181 theoretical results, it is easily recovered by replacing each θ_e with $\gamma\theta_e$.

182 **The connection Laplacian.** To a graph endowed with a unitary connection, we associate
 183 the *connection Laplacian* $L_\theta : \mathbf{C}^{\mathcal{V}} \rightarrow \mathbf{C}^{\mathcal{V}}$, defined by $L_\theta = D - A_\theta$, with A_θ a connection-
 184 aware adjacency matrix such that $(A_\theta)_{i,j} = e^{-i\theta_{(i,j)}}$ if $\{i, j\} \in \mathcal{E}$, and $A_{i,j} = 0$ otherwise. This

operator generalizes the Laplacian to non-trivial connections. Its quadratic form reads:

$$(1.2) \quad \langle f, \mathbf{L}_\theta f \rangle = \frac{1}{2} \sum_{e \in \vec{\mathcal{E}}} |f(t_e) - e^{\iota \theta_e} f(s_e)|^2.$$

For the trivial connection, with $\psi_e = \text{id}_{\mathbf{C}_{s_e}, \mathbf{C}_{t_e}}$ (i.e. $\theta_e = 0$) for all edges $e \in \vec{\mathcal{E}}$, we recover $\mathbf{L}_\theta = \mathbf{L}$ and the expression in Eq. (1.1). Unlike this specific case, $\langle f, \mathbf{L}_\theta f \rangle$ also penalizes functions that are incoherent with respect to the connection, including constant functions.

Let us now answer a natural question: when is the kernel $\ker \mathbf{L}_\theta$ non-empty, and what are the functions $f \in \mathbf{C}^\mathcal{V}$ that are not penalized by $\langle f, \mathbf{L}_\theta f \rangle$?

(P1') Angular synchronization and $\ker \mathbf{L}_\theta$. When the entries f_i of the vector f are *unitary* complex numbers $f_i = e^{\iota s_i}$, we recover from Eq. (1.2) the expression in Eq. (0.3). In fact, one obtains $|f(j) - e^{\iota \theta_{(i,j)}} f(i)|^2 = 2 - 2\Re((e^{\iota s_j})^* e^{\iota \theta_{(i,j)}} e^{\iota s_i})$, which yields:

$$(1.3) \quad \langle f, \mathbf{L}_\theta f \rangle = \sum_{\{i,j\} \in E} 2 - 2 \cos((s_j - s_i) - \theta_{(i,j)}).$$

Even though this equality only holds for $f \in U(\mathbf{C})^\mathcal{V}$, note that $\langle f, \mathbf{L}_\theta f \rangle = 0$ if and only if $f_i = e^{\iota \theta_{(j,i)}} f_j$ for all $(i, j) \in \vec{\mathcal{E}}$, that is, if $s_i = s_j + \theta_{(j,i)}$. As a consequence, $\mathbf{L}_\theta$ is not only semi-definite positive (as seen from Eq. (1.2)), but *positive* definite, *unless* there is an *exact* solution $x \in U(\mathbf{C})^\mathcal{V}$ to the associated angular synchronization problem, in which case the kernel $\ker \mathbf{L}_\theta$ is generated by x .

This behavior of the smallest eigenpair of $\mathbf{L}_\theta$ is in clear contrast to that of $\mathbf{L}$ (which is *always* zero, see property (P1)), and we explain in Section 2 how a generalization of property (P2) pertaining to *random propagations* can be used to solve connection-Laplacian-based problems.

2. Random Estimators for Tikhonov Smoothing under a Connection. This section shows how to smooth a complex signal on a graph via random propagations. Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a graph endowed with a connection Ψ . We aim at smoothing a signal $g \in \mathbf{C}^\mathcal{V}$ by solving:

$$(2.1) \quad f_* = \underset{f \in \mathbf{C}^\mathcal{V}}{\text{argmin}} \quad q \|f - g\|_2^2 + \langle f, \mathbf{L}_\theta f \rangle,$$

with $q \in \mathbf{R}_+^*$. This is nothing but a re-writing of Eq. (0.1). Note that $\langle f, \mathbf{L}_\theta f \rangle$ penalizes functions that are not coherent with respect to the connection Ψ .

Proposition 2.1. *The solution to Problem (2.1) can be expressed as:*

$$(2.2) \quad f_* = q(\mathbf{L}_\theta + q\mathbf{I})^{-1}g.$$

The proof, based on standard **CR**-calculus, is in Section 8 of the Supplementary Material. Let us now describe how to estimate f_* via local propagation over random paths on $\mathcal{G}$.

2.1. Local Estimation: a Feynman-Kac Formula. Feynman-Kac formulas express solutions of variational problems as the expectations of stochastic processes: we develop here an instance tailored to the Tikhonov smoothing problem of Eq. (2.1). We start with a few

Algorithm 2.1 Feynman-Kac estimator (Proposition 2.2).

- 1: $f_i \leftarrow 0$
 - 2: **Repeat** m times
 - 3: Sample a path $p \in P_i^\Gamma$ from ν_i $\triangleright$ By running an interrupted random walk on $\mathcal{G}$
 - 4: $f_i \leftarrow f_i + \psi_{p_\Gamma^*}(g_j)$ $\triangleright j$ the last node before interruption of p
 - 5: **Output** $\frac{1}{m}f_i$ $\triangleright$ An unbiased estimator of $f_*(i)$
-

definitions. A *path* p in $\mathcal{G}$ is an ordered sequence of oriented edges in $\vec{\mathcal{E}}$. We consider the two operations of concatenation pq of two paths p and q and of orientation-reversal p^* of a path p , and denote by P_i^j the set of paths from i to j in $\mathcal{G}$. Connection maps extend to paths inductively as $\psi_{pq} = \psi_q \circ \psi_p$, and we will for instance frequently encounter the map $\psi_{p^*} : \mathbf{C}_j \rightarrow \mathbf{C}_i$, acting by composition of the (inverse) rotations encountered along path $p \in P_i^j$ from i to j .

The stochastic process we consider is a random walk with a non-zero probability of being interrupted at any node. It is conveniently defined on an extended graph $\mathcal{G}^\Gamma$, with nodes $\mathcal{V} \cup \{\Gamma\}$ and edges $\mathcal{E} \cup \bigcup_{v \in \mathcal{V}} \{\{v, \Gamma\}\}$ (so that all the nodes of $\mathcal{V}$ are connected to Γ), where Γ is an additional *boundary* node. We then define the random walk $(u_t)_{t \geq 0}$, which starts at some fixed node $u_0 = i \in V$ and transitions to some other node at time t as follows:

- $u_{t+1} = \Gamma$ with probability $\frac{q}{d_{u_t} + q}$ (recall that d_{u_t} stands for the degree of node u_t),
- $u_{t+1} = v \in \mathcal{V}$ with probability $\frac{A_{u_t, v}}{d_{u_t} + q}$.

The process ends upon reaching node Γ . In other words, this is the natural random walk on $\mathcal{G}_\Gamma$, with a stopping criterion at the boundary Γ . Writing $u_l = \Gamma$ for some time l , this process results in a random path $((u_0, u_1), \dots, (u_{l-1}, u_l))$. We denote by P_i^Γ the set of all possible paths obtainable via this process, and by ν_i the resulting probability measure on P_i^Γ .

We can then show that propagating along paths sampled according to ν_i converges in expectation to the solution of Problem (2.1) on node i . More precisely:

Proposition 2.2. *Denoting by $j = u_{l-1}$ the last node reached before p reaches Γ , we have:*

$$(2.3) \quad f_*(i) = \mathbf{E}_{p \sim \nu_i} \left(\psi_{p_\Gamma^*}(g_j) \right),$$

where $p = p_\Gamma e_\Gamma$ with e_Γ the last edge in p .

In other words, an unbiased estimate of $f_*(i)$ is obtained by drawing a random path from i to Γ , and retropropagating the value of g at the node just before Γ (j in Proposition 2.2), taking into account the rotations along the path. See Alg. 2.1. This formula is a discrete-time analog of a Theorem in [40], and we generalize it to weighted graphs in the Supplementary Material (Section 8, Proposition 8.1), building upon a connection-aware version of property (P2).

Alg. 2.1 is simple but may be computationally expensive: estimation on one node requires sampling m paths, which needs to be repeated for each of the $|\mathcal{V}|$ nodes in $\mathcal{G}$ (a locality issue inherent to all Walk-on-Spheres-type algorithms [56]). We address these limitations in the remainder of this section and propose novel propagation-based estimators of f_* , allowing to update the Monte-Carlo estimates on *all the nodes at once*, by i) sampling and ii) propagating the signal, on specific subgraphs: the so-called MTSFs.

Remark 2.3. The restriction to complex signals on $\mathcal{G}$ and unitary connections (encoding 2D rotations) is mostly artificial, and Proposition 2.2 generalizes to transformations in e.g. $O(\mathbf{R}^d)$. This setting is encountered for instance for 3D molecule alignment in cryogenic electron microscopy (cryo-EM) [76], or in structure-from-motion problems [2], where the ψ_e 's act as 3D rotations (i.e. $\psi_e \in SO(\mathbf{R}^3)$).

2.2. Multi-Type Spanning Forests: a first global estimator. To build our global estimators, we rely on decompositions of the graph into *Multi-Type Spanning Forests* (MTSFs) [22]. A MTSF $\phi \subseteq \mathcal{V} \cup \mathcal{E}$ decomposes $\mathcal{G}$ into disjoint components that are either *rooted trees*, or *unicycles*. More precisely, a MTSF ϕ must have fixed cardinality $|\phi| = |\mathcal{V}|$, and is divided into (maximal) components $c_\phi \subseteq \phi$, which must be either:

- a rooted tree $c_\phi \subseteq \mathcal{V} \cup \mathcal{E}$, such that $c_\phi \cap \mathcal{E}$ is a connected cycle-free subset of edges, and $c_\phi \cap \mathcal{V}$ is reduced to a single node, called *root*, connected to $c_\phi \cap \mathcal{E}$;
- a unicycle $c_\phi \subseteq \mathcal{E}$, which is a connected subset of edges containing *exactly* one cycle.

See Figure 2.1 for an illustration. Note that in the absence of unicycles, a MTSF is a *rooted spanning forest* [3, 4], whereas a tree-free MTSF is a *spanning forest of unicycles* [41].

Denote by $\mathcal{M}(\mathcal{G})$ the set of all MTSFs over $\mathcal{G}$. Consider the distribution $\mathcal{D}_{\mathcal{M}}$ over $\mathcal{M}(\mathcal{G})$:

$$(2.4) \quad \mathbf{P}_{\mathcal{D}_{\mathcal{M}}}(\phi) \propto q^{|\phi \cap \mathcal{V}|} \prod_{C \in \mathcal{C}(\phi)} (2 - 2 \cos(\theta_C)),$$

where $\mathcal{C}(\phi)$ is the set of cycles belonging to the unicycles of ϕ , and θ_C is the argument of the unitary complex number associated to the connection map ψ_C obtained from a path traversing C *one time* (as in Fig. 1.1). Note that, while ψ_C depends on the path's orientation, $\cos(\theta_C)$ is insensitive to this orientation. Also, a cycle has a non-zero probability of being sampled if and only if it is *inconsistent* (if C is perfectly consistent: $\theta_C = 0$ and $2 - 2 \cos(\theta_C) = 0$). $\mathcal{D}_{\mathcal{M}}$ is a rooted variant of the distribution considered in [22], and is a DPP⁶ over the set $\mathcal{V} \cup \mathcal{E}$. We can now state our main theorem.

Theorem 2.4. Let ϕ be a MTSF sampled according to $\mathcal{D}_{\mathcal{M}}$. Denote by $a \xrightarrow{\phi} b$ the unique path from a to b in some tree of a MTSF ϕ and, if $v \in \mathcal{V}$ belongs to a tree of ϕ , by $r_\phi(v)$ the root of this tree. Consider the estimator

$$(2.5) \quad \tilde{f}(i, \phi, g) \begin{cases} = \psi_{r_\phi(i) \xrightarrow{\phi} i}(g(r_\phi(i))) & \text{if } i \text{ belongs to a tree} \\ = 0 & \text{if } i \text{ lies in a unicycle.} \end{cases}$$

When i belongs to a tree, this estimator propagates the value of the signal g from the root $r_\phi(i)$ to i , via the connections. When i belongs to a unicycle, it is simply zero. Then, one has:

$$(2.6) \quad f_*(i) = \mathbf{E}_{\mathcal{D}_{\mathcal{M}}}(\tilde{f}(i, \phi, g)),$$

i.e., the estimator is unbiased. Moreover, the variance of $\tilde{f}$ can be characterized as⁷

$$(2.7) \quad \mathbf{E}_{\mathcal{D}_{\mathcal{M}}}(\|\tilde{f}(\phi, g) - f_*\|^2) = \langle g, (\mathbf{I} - \mathbf{K}^2)g \rangle,$$

⁶So is the distribution of [22], but their non-combinatorial argument does not carry over to our rooted process. See the Section 9 of the Supplementary Material for a proof in this case *proof of what?*

⁷Here, the variance of a complex-valued random variable $\tilde{z}$ is defined as $\text{var}(\tilde{z}) = \mathbf{E}(|\tilde{z}|^2) - |\mathbf{E}(\tilde{z})|^2$, and we compute in Eq. (2.7) the expected squared error $\mathbf{E}_{\mathcal{D}_{\mathcal{M}}}(\|\tilde{f}(\phi, g) - f_*\|^2) = \sum_{i \in \mathcal{V}} \text{var}(\tilde{f}(i, \phi, g))$.

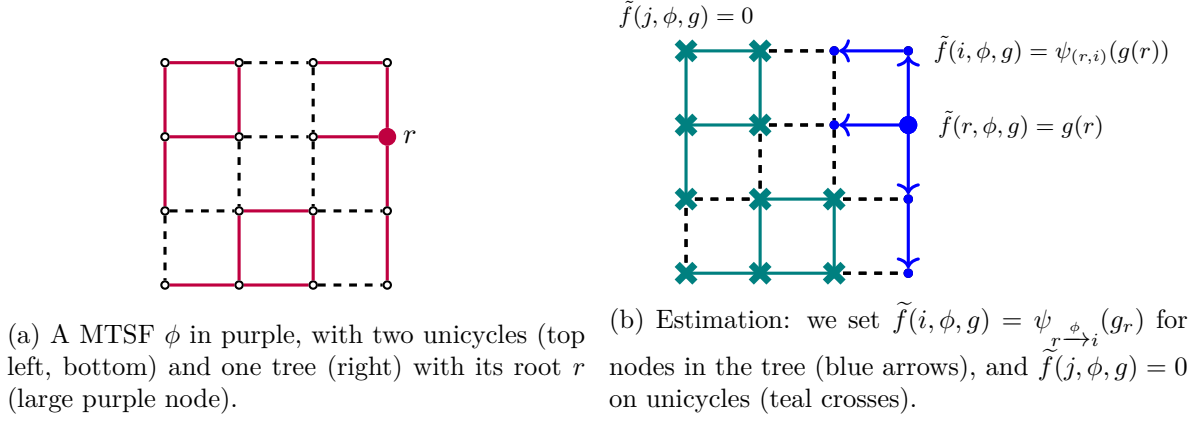


Figure 2.1: Theorem 2.4 illustrated on the 4×4 grid graph, on one MTSF.

where $\mathbf{K} = q(\mathbf{L}_{\theta} + q\mathbf{I})^{-1}$ denotes the connection-aware Tikhonov smoothing operator.

The proof of Theorem 2.4 relies on the reformulation of $\mathcal{D}_{\mathcal{M}}$ as a DPP, and an extension of arguments coming from both [41] and [61]. It is a special case of Theorem 9.1, which includes weighted graphs, and is proved in Section 9 of the Supplementary Material.

In a less technical phrasing, propagating for each tree in a MTSF ϕ the value of g at its root r to its other nodes i (along path $r \xrightarrow{\phi} i$) results in an unbiased estimator of f_* . The estimation on nodes belonging to unicycles is then 0. See Alg. 2.2 and Figure 2.1.

Algorithm 2.2 MTSF-based estimator (Theorem 2.4).

```

1:  $f_i \leftarrow 0 \ \forall i \in \mathcal{V}$ 
2: Repeat  $m$  times
3:   Sample  $\phi \in \mathcal{M}(\mathcal{G})$  from  $\mathcal{D}_{\mathcal{M}}$  ▷ Alg. 2.3 can be used
4:   for  $i \in \mathcal{V}$  do
5:     if  $i$  belongs to a rooted tree of  $\phi$  then
6:        $f_i \leftarrow f_i + \psi_{r_{\phi(i)} \rightarrow i}^{\phi}(g(r_{\phi(i)}))$ 
7:     end if
8:   end for
9: Output  $\frac{1}{m}(f_1, \dots, f_{|\mathcal{V}|})$ 

```

Remark 2.5. If the connection Ψ is trivial (i.e., $\psi_e = \text{id}_{\mathbf{C}_{s_e}, \mathbf{C}_{t_e}}$ for all edges), we recover the result from [61]. There are no inconsistent cycles in this case, and $\mathcal{D}_{\mathcal{M}}$ reduces to another determinantal distribution $\mathcal{D}_{\mathcal{F}}$ over rooted spanning forests $\phi \in \mathcal{V} \cup \mathcal{E}$ [3, 4].

Unlike Alg. 2.1, Alg. 2.2 allows to update the estimates on *all the nodes* for each sampled MTSF, in $\mathcal{O}(|\mathcal{V}|)$ time. On the other hand, it becomes necessary to sample a MTSF ϕ

according to $\mathcal{D}_{\mathcal{M}}$. A first naive strategy is to use algorithms designed to sample generic DPPs. They however rely on the eigendecomposition of a kernel matrix of size $(|V|+|E|) \times (|V|+|E|)$, which is prohibitive. In the next section, we show how to take advantage of a very efficient random-walk-based algorithm (from [22]), and analyze its expected computational cost.

2.3. Sampling Multi-Type Spanning Forests. Efficient sampling from $\mathcal{D}_{\mathcal{M}}$ is possible via *loop-erased random walks*, traditionally used in Wilson’s algorithm to sample uniform spanning trees [82]. The simplest scenario arises when the following condition is verified⁸.

Condition 2.6 (Weak-Inconsistency). *For all cycles C in $\mathcal{G}$, $\cos(\theta_C) \geq 0$.*

We stress that this condition constrains *all* the cycles of the graph, and not just the cycles $\mathcal{C}(\phi)$ in some MTSF $\phi \in \mathcal{M}(G)$. If Condition 2.6 is satisfied, $1 - \cos(\theta_C) \leq 1$ defines a probability measure over the oriented cycles in G , which can be leveraged to efficiently sample an MTSF ϕ , as we detail in the following. If this condition does not hold, one can still efficiently sample MTSFs, by relying on importance sampling, as explained later in this section.

MTSF sampling algorithm. The algorithm of [22], recalled here as Alg. 2.3, samples a MTSF ϕ according to $\mathcal{D}_{\mathcal{M}}$ by simulating multiple random walks on $\mathcal{G}^{\Gamma}$, and constructs ϕ iteratively from these random paths. The precise way in which a path is turned into a part of ϕ relies on a *loop-erasure* procedure. The full sampling scheme is detailed in Alg. 2.3, performing random walks that can be stopped in a number of ways: by being interrupted (*i.e.* reaching Γ), by building a cycle θ_C (kept with probability $1 - \cos(\theta_C)$), or by reaching a node already spanned by ϕ ⁹. We suppose that the nodes of $\mathcal{G}$ are arbitrarily ordered in a queue, and say that a node i is spanned by ϕ if $i \in \phi$ or if i is an endpoint of some edge $e \in \phi$. We also use a function `random_successor`(u) that, at each call, randomly outputs either Γ with probability $\frac{q}{d_u+q}$, or some node v with probability $\frac{A_{u,v}}{d_u+q}$.

One then proves the following by applying the arguments of [22] (which describes how to sample *unrooted* MTSFs), and keeping track of the roots.

Proposition 2.7. *Suppose that Condition 2.6 holds. Then, Alg. 2.3 outputs a MTSF ϕ distributed according to $\mathcal{D}_{\mathcal{M}}$.*

Let us now discuss the cost of Alg. 2.3.

Proposition 2.8. *The expected number of calls to `random_successor` (*i.e.*, the expected number of random walk steps), in Alg. 2.3, is upper-bounded by*

$$(2.8) \quad \text{tr} \left((\mathbf{L} + q\mathbf{I})^{-1} (\mathbf{D} + q\mathbf{I}) \right).$$

Also, Alg. 2.3 can be implemented with an expected time complexity upper-bounded by $\mathcal{O} \left(\frac{|\mathcal{E}|}{q} \right)$.

⁸Condition 2.6 has already been identified as a technical sampling condition for Wilson-like algorithms (in [39] and [22]). It also appears in a different guise in [40], under the name of *trace-positivity*, where it allows to simplify some technical statements.

⁹Some implementation details of Alg. 2.3 are not made completely explicit here, such as the precise way in which we detect cycles (instruction at line 11), or how to track the cycle-acceptance probabilities (*i.e.* θ_C). Please refer to Section 10 of the Supplementary Material and our Julia implementation for more details

Algorithm 2.3 MTSF sampling algorithm [22].

```

1:  $\phi \leftarrow \emptyset$ 
2: while  $\phi$  not spanning do
3:   Let  $i \in \mathcal{V}$  be the first node in the queue not spanned by  $\phi$ 
4:    $u \leftarrow i$   $\triangleright u$  is the current node of the random walk
5:    $p \leftarrow \epsilon$   $\triangleright \epsilon$  the empty path
6:   while ( $u \neq \Gamma$ ) and ( $p$  does not intersect  $\phi$  or contain a cycle) do
7:      $u' \leftarrow \text{random\_successor}(u)$   $\triangleright$  Move to next node
8:     if  $u' \neq \Gamma$  then
9:        $e \leftarrow (u, u')$ ,  $p \leftarrow pe$   $\triangleright$  Add  $e$  to the path  $p$ 
10:    end if
11:    if  $p$  contains a cycle  $C$  then
12:      Remove this cycle from  $p$  with probability  $\cos(\theta_C)$ 
13:    end if
14:     $u_{\text{old}} \leftarrow u$ ,  $u \leftarrow u'$   $\triangleright u_{\text{old}}$  the previous node
15:  end while
16:  if  $u = \Gamma$  then
17:     $\phi \leftarrow \phi \cup p \cup u_{\text{old}}$   $\triangleright$  Add the sampled path  $p$  and the root  $u_{\text{old}}$  to  $\phi$ 10
18:  else
19:     $\phi \leftarrow \phi \cup p$ 
20:  end if
21: end while
22: Output  $\phi$ 

```

See Section 10 of the Supplementary Material for the proof. The bound is obtained by noting that Alg. 2.3 necessitates *fewer* steps than the Wilson-like algorithm in [61], used to sample random spanning forests [3,4], with expected number of steps exactly given by (2.8). Unlike the algorithm in [61] though, Alg. 2.3 requires both tracking of the angular offsets accumulated along a cycle, and detecting said cycle, at additional computational cost. We discuss two possible implementations in Section 10 of the Supplementary Material, one of them resulting in the $\mathcal{O}\left(\frac{|\mathcal{E}|}{q}\right)$ expected *time complexity* mentioned in Proposition 2.8. This translates to $\mathcal{O}\left(\frac{|\mathcal{E}|}{q} + |\mathcal{V}|\right)$ expected runtime for Alg. 2.2.

When Condition 2.6 does not hold: Importance Sampling. In case Condition 2.6 is not satisfied, one can still estimate f_* using an importance sampling strategy¹¹. Specifically, one can threshold the incoherence $2 - 2\cos(\theta_C)$ and sample from

$$(2.9) \quad \mathbf{P}_{IS}(\phi) \propto q^{|\phi \cap \mathcal{V}|} \prod_{C \in \mathcal{C}(\phi)} \min(2, 2 - 2\cos(\theta_C))$$

¹⁰We abuse notations and denote here by p the set of *non-oriented* edges in the path.

¹¹The importance weights we consider have been proposed in [22], in the context of spectral sparsification.

using a variant of Alg. 2.3, where a cycle is systematically kept, at line 12, if $\cos(\theta_C) \leq 0$. The estimation then uses the importance weights

$$(2.10) \quad w(\phi) = \prod_{C \in \mathcal{C}(\phi)} \max(1, 1 - \cos(\theta_C)),$$

which allows to estimate cf_* for some unknown constant $c > 0$. Note that in the angular-synchronization pipeline we consider in Section 4, this is sufficient as we only need an estimation of f_* up to a multiplicative constant. For smoothing purposes, one can further consider a self-normalized importance-sampling estimator [58]: for m MTSFs $\{\phi_k\}_{k=1}^m$ sampled according to the distribution of Eq. (2.9), we set

$$(2.11) \quad \tilde{f}_{IS} = \frac{1}{\sum_{k=1}^m w(\phi_k)} \sum_{k=1}^m \tilde{f}(\phi_k, g),$$

which tends to f_* as $m \rightarrow \infty$. This result holds regardless of the incoherence of the connection, but the estimator is only guaranteed to be unbiased in the limit.

Both the Feynman-Kac-based Alg. 2.1 and the MTSF-based Alg. 2.2 can be roughly described as performing a random walk on $\mathcal{G}$ before stopping at some root node, and then retro-propagating the value from this root to the starting point of the random walk. The main difference is that MTSFs allow to update the estimation on all nodes jointly. We will now see how our estimator $\tilde{f}$ can be improved by implementing two variance reduction techniques.

2.4. Variance Reduction for the MTSF-based estimator. A paramount property of Monte-Carlo estimators is not only their unbiased behavior in expectation, but also their variance: they are significantly improved when used in conjunction with efficient variance reduction techniques. We propose two such improvements over $\tilde{f}$, based on the classical approaches of *Rao-Blackwellization* [7, 66] and *control variates* [42] respectively, generalizing the variance-reduction techniques introduced in [61, 63].

Rao-Blackwellization. Rao-Blackwellization leverages the two laws of total expectation and variance, roughly stating that conditioning an estimator using another statistic extracted from the same sample still results in an unbiased estimator, with lower variance (see Section 11 of the Supplementary Material for more details in our case). Here, our technique consists in conditioning on the set of edges in ϕ . This yields the following estimator:

$$(2.12) \quad \bar{f}(i, \phi, g) = \psi_{r_\phi(i) \xrightarrow{\phi} i} (h_\phi(r_\phi(i), g)) \text{ if } i \text{ belongs to a rooted tree,}$$

where h_ϕ is defined as

$$(2.13) \quad h_\phi(r, g) = \frac{\sum_{j \in c_\phi(r)} \psi_{j \xrightarrow{\phi} r} g(j)}{|c_\phi(r)|},$$

and $c_\phi(r)$ is the set of nodes spanned by the tree containing r . If i belongs to a unicycle, we once again set $\bar{f}(i, \phi, g) = 0$. This results in Alg. 2.4, which amounts to:

- Computing at the root r of each tree the average $h_\phi(r, g)$ of the values of g over the tree (obtained by propagating from each node i in the tree to r).

Algorithm 2.4 Rao-Blackwellized MTSF-based estimator (Theorem 2.4).

```

1:  $f_i = 0 \ \forall i \in \mathcal{V}, h_r = 0 \ \forall r \in \mathcal{V}$ 
2: Repeat  $m$  times
3:   Sample  $\phi \in \mathcal{M}(\mathcal{G})$  from  $\mathcal{D}_{\mathcal{M}}$  ▷ Via Alg. 2.3
4:   for  $r \in \phi \cap \mathcal{V}$  do
5:     for  $j \in c_{\phi}(r)$  do
6:        $h_r = h_r + \psi_{j \xrightarrow{\phi} r} g(j)$  ▷ Propagate and average at the root
7:     end for
8:   end for
9:   for  $i \in \mathcal{V}$  belonging to a rooted tree of  $\phi$  do
10:     $f_i = f_i + \psi_{r_{\phi}(i) \xrightarrow{\phi} i} (h_{r_{\phi}(i)})$  ▷ Propagate back
11:  end for
12: Output  $\frac{1}{m}(f_1, \dots, f_{|\mathcal{V}|})$ 

```

- Propagating this average back to the other nodes of the tree.

As compared with Alg. 2.2, this procedure can be implemented at little additional cost¹².

Control variates. Second is the introduction of control variates: an addition of another random quantity to $\bar{f}$, designed to have zero mean (so that the expectation remains unchanged), but resulting in an estimator with lower variance when designed properly. We propose to use a single *gradient-descent* step with parametrized step-size α :

$$(2.14) \quad \hat{f}(\phi, g) = \bar{f}(\phi, g) - \alpha \mathbf{P}(q^{-1}(\mathbf{L}_{\theta} + q\mathbf{I})\bar{f}(\phi, g) - g),$$

where $\mathbf{P} = (q^{-1}\mathbf{D} + \mathbf{I})^{-1}$ is a diagonal preconditioner for the system $q^{-1}(\mathbf{L}_{\theta} + q\mathbf{I})f = g$, and $\alpha \in \mathbf{R}_{+}^{*}$. A good choice of step-size α is crucial in order to obtain a significant reduction of variance. We simply take $\alpha = 1$ in the following (see Section 11 of the Supplementary Material for empirical results backing this choice).

We prove in Section 11 of the Supplementary Material that generalizations of both $\bar{f}$ and $\hat{f}$ are unbiased estimators of f_* , along with some results on the concentration of $\bar{f}$.

Proposition 2.9. *The variance-reduced estimators are unbiased:*

$$(2.15) \quad \mathbf{E}_{\mathcal{D}_{\mathcal{M}}}(\bar{f}(i, \phi, g)) = \mathbf{E}_{\mathcal{D}_{\mathcal{M}}}(\hat{f}(i, \phi, g)) = f_*(i).$$

Further, the variance of the Rao-Blackwellized estimator is given by

$$(2.16) \quad \mathbf{E}_{\phi \sim \mathcal{D}_{\mathcal{M}}}(\|\bar{f}(\phi, g) - f_*\|^2) = \langle g, (\mathbf{K} - \mathbf{K}^2)g \rangle,$$

where $\mathbf{K} = q(\mathbf{L}_{\theta} + q\mathbf{I})^{-1}$, and we have the following finite-sample concentration bound. Let $\varepsilon, \delta \in (0, 1)$, and consider sampling m MTSFs $\{\phi_k\}_{k=1}^m$. Then, provided that¹³

$$(2.17) \quad m \geq \frac{6}{\varepsilon^2} \log \left(\frac{|\mathcal{V}|}{\delta} \right),$$

¹²Note also the similarity to the Belief Propagation algorithm over trees [54].

¹³The factor 6 in Eq. (11.5) is not optimal: it is obtained from a more explicit bound appearing in the proof.

it holds that

$$(2.18) \quad \mathbf{P} \left(\forall g \in \mathcal{C}^{\mathcal{V}}, \quad \left\| \left(\frac{1}{m} \sum_{k=1}^m \bar{f}(\phi_k, g) \right) - f_* \right\|_2 \leq \varepsilon \|g\|_2 \right) \geq 1 - \delta.$$

Both variance-reduction techniques are easy to implement, and do not incur large additional computational costs: the Rao-Blackwellization $\bar{f}$ comes at $\mathcal{O}(|\mathcal{V}|)$ additional cost, and the gradient-descent step in $\hat{f}$ entails a *single* matrix-vector multiplication, in $\mathcal{O}(|\mathcal{E}|)$ time. Further, Eq. (11.6) shows that the number of forests required to reach a fixed precision grows *logarithmically* with the size of the graph, and that it is *insensitive to the choice of q and to the incoherence of the graph* (which both affect the conditioning of the system, and the performance of conjugate-gradient solvers¹⁴). Note though that these parameters *do* impact the sampling cost for MTSFs.

3. Numerical Results under Weak-Inconsistency. We now analyze the behavior of the estimator proposed in Theorem 2.4, along with the improved versions discussed in Section 2.4, and compare their performance with (deterministic) conjugate-gradient-based solvers [67], on graphs with different topologies¹⁵. We perform experiments on the following connection model.

Connection Model. For a given graph $\mathcal{G}$ with n nodes, we associate to each node v an angle ω_v chosen uniformly at random in $[0, 2\pi)$, and we set

$$(3.1) \quad \theta_e = \omega_{t_e} - \omega_{s_e} + \eta \varepsilon_e$$

for all edges e , with ε_e a perturbation uniformly distributed in $[-1, 1]$, and $\eta \in \mathbf{R}_+^*$ a scaling constant. We set $\eta = \frac{\pi}{2n}$ to ensure that Condition 2.6 is satisfied (so that Alg. 2.3 in fact samples a MTSF according to $\mathcal{D}_{\mathcal{M}}$).

3.1. A First Runtime Experiment on Erdős-Rényi Graphs. We first analyze the behavior of our estimator $\tilde{f}$ with respect to two parameters: the choice of regularization parameter q and the mean degree $\bar{d}$ of the graph (controlling the graph's density). In our first experiment, we let $\bar{d}$ take values in $\{50, 100, 150, 200\}$, and q take values equal to $q'\bar{d}$, with $q' \in \{10^{-3}, 10^{-1}, 1\}$. The complexity bound of Eq. (2.8) becomes *linear in $|\mathcal{V}|$* when using such a parametrization.

Setup. We generate Erdős-Rényi random graphs $\mathcal{G} \sim \text{ER}(n, p)$ of size $n = 10^4$ for varying $p \in [0, 1]$, so that each edge e independently appears with probability p . To control the density, we set $p = \frac{\bar{d}}{n-1}$ so that the expected mean degree of these random graphs is $\bar{d}$. For each such graph, we generate a random signal $f \in \mathbf{C}^{\mathcal{V}}$ with independent complex Gaussian entries $f_v \sim \mathcal{N}_{\mathbf{C}}(0, 1)$.

We then measure the running time of sampling *one* MTSF ϕ and applying the estimator $\tilde{f}$. As a reference, we also measure in the same manner the runtime of the matrix-vector multiplication $\mathbf{L}_{\theta} f$, where $\mathbf{L}_{\theta}$ is implemented as a sparse Hermitian matrix in CSC format. Note that this operation is the most expensive part of an iteration of the Conjugate-Gradient algorithm, and serves as a simple baseline to compare computation costs.

¹⁴See the *Discussion* paragraph of Section 3.2 for a reminder on the expected performance of CG

¹⁵We perform all our measurements using a *single thread* on a laptop with an intel i7-1185G7 processor.

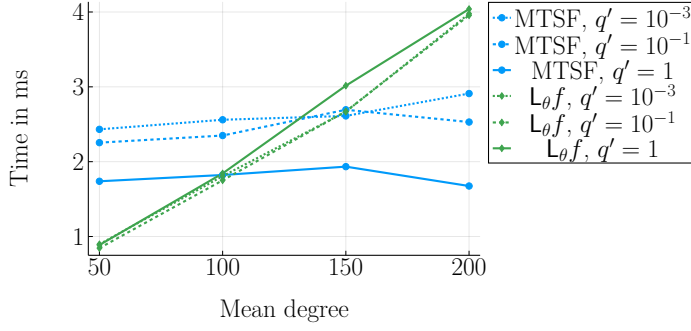


Figure 3.1: Runtime when varying $\bar{d}$, for different values of q .

The results, depicted in Figure 3.1, are the average over 10^4 time measurements, themselves averaged over 10 realizations of the graph G .

Analysis. Two trends emerge in Figure 3.1. First, as q' (hence q) increases, computing $\tilde{f}$ becomes less expensive, since the probability of the random walk stopping at some root node increases. Second, in contrast to the matrix-vector product $L_\theta f$, our estimator is less sensitive to the density of the graph: computing $\tilde{f}$ is more expensive than computing $L_\theta f$ for $\bar{d} = 50$, and systematically faster for $\bar{d} = 200$. This reflects the $\mathcal{O}(|\mathcal{V}|)$ expected complexity for $\tilde{f}$ (obtained with the parametrization $q = q'\bar{d}$)¹⁶.

3.2. Runtime-Precision Trade-offs in the context of Complex Graph Signal Denoising.

We compare the performance of our improved estimators $\tilde{f}$ and $\hat{f}$ with conjugate-gradient methods [67]. The objective is to recover a signal $f_\top \in \mathbf{C}^\mathcal{V}$ given a noisy degradation $g = f_\top + \varepsilon$.

In the specific instance where $f_\top \in \mathbf{R}^\mathcal{V}$ and the connection on $\mathcal{G}$ is trivial ($\psi_e = \text{id}_{\mathbf{C}_{s_e}, \mathbf{C}_{t_e}}$ for all $e \in \mathcal{E}$), a common assumption in the graph signal processing literature is that $f_\top$ is a *smooth* signal, that is, a linear combination of the first (low-frequency) eigenvectors of L (associated to the smallest eigenvalues) [50, 64].

We here similarly assume that $f_\top \in \mathbf{C}^\mathcal{V}$ is B -bandlimited (*i.e.* $f_\top \in \text{span}(u_1, \dots, u_B)$, with u_i the i -th eigenvector of L_θ), so that solving the Tikhonov smoothing Problem (2.1):

$$(3.2) \quad \underset{f \in \mathbf{C}^\mathcal{V}}{\text{argmin}} \quad q\|f - g\|_2^2 + \langle f, L_\theta f \rangle,$$

should allow to faithfully recover $f_\top$ from g , by penalizing high-frequency components in the optimal solution $f_* = q(L_\theta + qI)^{-1}g$. We refer the reader to Section 13 of the Supplementary Material for additional supporting arguments in this connection-aware setting.

Recall that this matrix inversion can be computed using a Cholesky decomposition (for an exact solution), or a conjugate-gradient-based iteration (for a high quality approximation). We compare our estimators with these methods, and consider two cost functions:

- The reconstruction error $e_r(f) = \|f - f_\top\|_2/n$, measuring the quality of the denoising.
- The approximation error $e_a(f) = \|f - f_*\|_2/n$, measuring the quality of the approximation of f_* .

¹⁶Results are similar when varying the size n of the graph (not shown).

Our estimators and the conjugate-gradient algorithms are respectively parametrized by the number of MTSFs used and the number of gradient steps, that we both denote by m .

Setup. For each graph, we generate a B -bandlimited signal $f_\top$, each u_i being weighted by a random complex Gaussian value $a_i \sim \mathcal{N}_{\mathbb{C}}(0, 1)$. We then degrade $f_\top$ with some additive Gaussian noise $\varepsilon \sim \mathcal{N}_{\mathbb{C}}(0, \sigma^2)$ (independently on each entry), with σ^2 such that the SNR is equal to 2, and determine the optimal parameter q_* for which $e_r(f_*) = e_r(q_*(\mathbf{L}_\theta + q_*\mathbf{I})^{-1}g)$ is minimized¹⁷, before measuring the errors and running time associated to different values of m (taken amongst 10 logarithmically-spaced values from 1 to 100). The iterative algorithms are initialized at g .

We compare the following estimation strategies: the estimators $\bar{f}$ and $\hat{f}$, the conjugate-gradient descent with no preconditioner, with a simple diagonal preconditioner $\mathbf{P} = (q^{-1}\mathbf{D} + \mathbf{I})^{-1}$, and with a CROUT ILU preconditioner¹⁸. Computing this high-quality ILU preconditioner is too expensive to be competitive with the other methods, and we only include these results on one type of graph, for illustration purposes.

Each runtime measurement is averaged over 100 runs. The results are averaged over 5 realizations of the noise ε and 10 samples for each random graph model. We plot in Figure 3.2 the mean results.

Graphs used. We use the following graphs:

- An ε -graph obtained by sampling $|\mathcal{V}| = 10^4$ i.i.d. points x_i 's in $[0, 1]^3$, with an edge between x_i and x_j whenever $\|x_i - x_j\|_2 < 0.1$.
- A graph generated from a *Stochastic Block Model* (SBM), with $|\mathcal{V}| = 10^4$ nodes, each belonging to one of two communities C_1 and C_2 of size 5000 each. In this model, an edge is drawn randomly between two nodes i, j with probability¹⁹ $\frac{c_{k,l}}{n}$, where k, l denote respectively the community label of i and j . Here, we take $c_{1,1} = c_{2,2} = 36$ and $c_{1,2} = c_{2,1} = 4$, resulting in an average degree $\bar{d} = 40$.
- A graph generated from a related *Degree-Corrected Stochastic Block Model* (DC-SBM 1), with two communities of size 5000 and an edge between nodes $i \in C_k$ and $j \in C_l$ present with probability $p_i p_j \frac{c_{k,l}}{n}$. p_i is a randomly sampled positive real value, representing the intrinsic *connectivity* of node i , with $\mathbf{E}(p_i) = 1$ and finite second moment [37]. Here, we take $c_{1,1} = c_{2,2} = 36$, $c_{1,2} = 4$, and the p_i 's are drawn from a normalized mixture of Gaussian distributions²⁰, resulting in a graph with mean degree $\bar{d}$ close to 40. The objective of adding this model is to illustrate, at constant density, how the degree distribution affects the results.
- Another DC-SBM-graph (DC-SBM 2), with higher density (typically with an average degree more than 10 times that of the previous DC-SBM model). We take two communities of size 5000, $c_{1,1} = c_{2,2} = 480$, $c_{1,2} = 20$, and the p_i 's are drawn from another

¹⁷We perform our search in $(0, 30)$.

¹⁸We use the implementations from the libraries: <https://github.com/JuliaLinearAlgebra/IterativeSolvers.jl>, <https://github.com/JuliaLinearAlgebra/Preconditioners.jl> and <https://github.com/haampie/IncompleteLU.jl>. For the ILU preconditioner, we fix the drop threshold to 0.1.

¹⁹It is usually assumed that $n > c_{k,l}$, so that we always have $\frac{c_{k,l}}{n} \in [0, 1]$. Most actual implementations use the probabilities $\min(\frac{c_{k,l}}{n}, 1)$ in case this assumption is not satisfied.

²⁰Specifically, the connectivity parameters are independently sampled from a mixture of $\mathcal{N}(50, 20)$, $\mathcal{N}(500, 100)$ and $\mathcal{N}(10000, 100)$, with weights of 0.59, 0.4 and 0.01 respectively, and then normalized.

Graph	$ \mathcal{V} $	$\bar{d}$	$d_{\min}$	$d_{\max}$	q_*
ε -graph	10000	37.3	5.2	67.5	6.518
SBM	10000	39.87	17.7	66.6	21.04
DC-SBM 1	9833.7	33.4	1	906.1	2.634
DC-SBM 2	9932.8	418.8	1	1513.7	4.31
AS CAIDA	26475	4.032	1	2628	0.4808

Table 3.1: Experimental parameters associated to each graph. Values are averaged over all 10 realizations for random graph models.

mixture of Gaussian distributions²¹. The objective of this second DC-SBM model is to illustrate how density affects the results.

- We also illustrate the results on a real-world graph: a relationship graph for internet Autonomous Systems (AS), recorded by the Center for Applied Internet Data Analysis (CAIDA) and provided in the SNAP datasets [48]²².

In the event of a randomly generated graph containing isolated nodes, we remove them so that the graph is connected (to make the interpretations simpler). We summarize in Table 3.1 the information concerning the graphs generated, and the associated optimal q_* 's (averaged over all realizations of the noise ε). All those graphs are endowed with a synthetic connection as specified in Eq. (3.1). The parameters used for the (DC) SBM models ensures that the resulting graphs have a strong community structure.

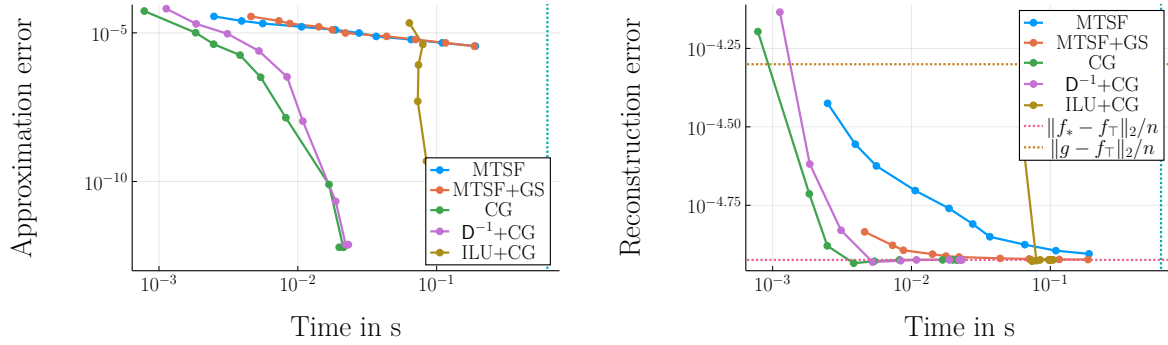
For the ε -graph and the AS-graph, we arbitrarily set $B = 5$ when generating the bandlimited signal $f_{\top}$. For (DC) SBMs, we take $B = 2$. It is known that the eigenvectors of the *combinatorial* Laplacian L encode the community structure of these graphs (here,, and taking $B = 2$ here results in signals coherent with the community structure (see Section 13 of the Supplementary Material for more on the first few eigenvectors of L_{θ}).

Discussion. Let us first comment on the approximation error for the ε -graph (Figure 3.2a, left). Both of our estimators converge linearly in log-log-scale with a mild slope, as expected for Monte-Carlo estimators. Any of the other three conjugate-gradient-based algorithms achieves a better approximation error within 3 steps than our estimators in 100 steps (as expected for Monte-Carlo convergence rates), and our methods are not competitive in this setting. However, in this denoising setting, the quality of the denoising is reflected in the reconstruction error (depicted in all the plots of Figure 3.2). Overall, we observe that:

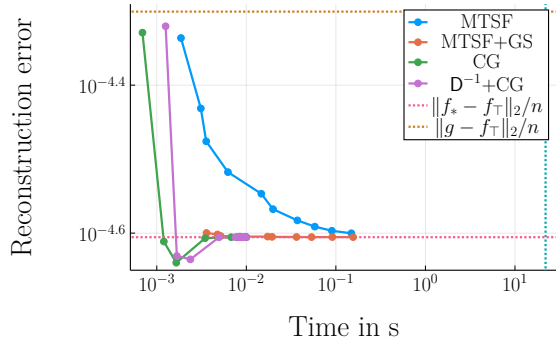
- (1) $\hat{f}$ consistently performs better than $\bar{f}$, but how much better depends on the graph (see *e.g.* SBM and DC-SBM 1), and improvements are smaller for graphs with more heterogeneous degree distributions (DC-SBM graphs);
- (2) compared with MTSF-based estimators, the CG algorithms perform *worse* on graphs with heterogeneous degree distributions;

²¹Here from the mixture of $\mathcal{N}(50, 20)$, $\mathcal{N}(1000, 50)$, $\mathcal{N}(5000, 100)$ and $\mathcal{N}(10000, 100)$, with weights of 0.45, 0.1, 0.44 and 0.01.

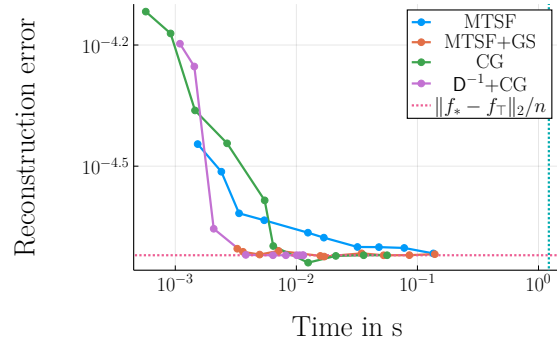
²²We use the library available at <https://github.com/JuliaGraphs/SNAPDatasets.jl>.



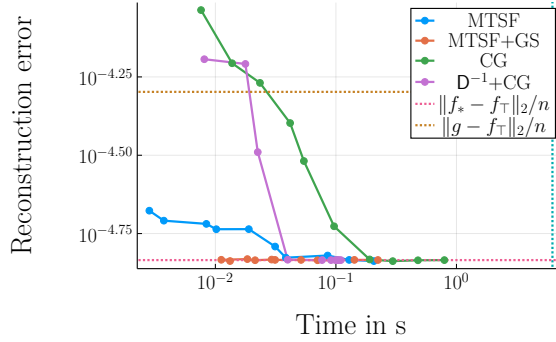
(a) ε -graph. Left: Approximation error. Right: Reconstruction error.



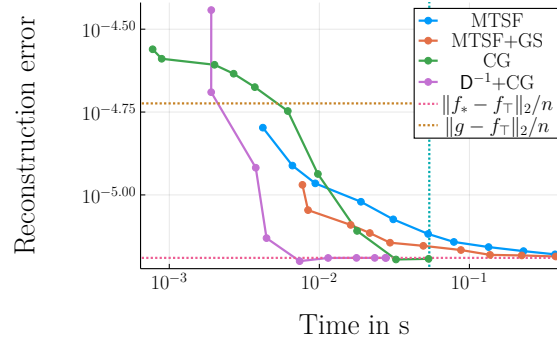
(b) SBM Reconstruction error.



(c) DC-SBM 1 Reconstruction error.



(d) DC-SBM 2 Reconstruction error.



(e) AS CAIDA Reconstruction error.

Figure 3.2: Runtime-precision trade-offs for Tikhonov smoothing with optimal q . Each datapoint corresponds to a value of m . The curve labelled MTSF corresponds to the estimator $\bar{f}$, and the curve labelled MTSF+GS to the estimator $\hat{f}$. The vertical blue line records the average runtime of the Cholesky-based solver, the horizontal red line the average reconstruction error of the exact solution to the Tikhonov problem, and the horizontal yellow line the average reconstruction error of the noisy signal. Vertical bars represent standard deviations.

(3) CG solvers perform worse than our methods when the density increases (MTSF-based methods reach optimal reconstruction error 10 times faster on DC-SBM 2).

The relatively poor performance of CG on DC-SBM graphs is not unexpected. CG is widely known to be quite sensitive to ill-conditioning [67], which is why problem-specific preconditioners are often necessary. For instance, one can show that for a linear system of the form $Ax = b$ with solution x_* (where A is self-adjoint and positive definite), the time-complexity required by CG to reach an approximation $\tilde{x}$ of x_* such that $\frac{\|\tilde{x} - x_*\|_A^2}{\|x_*\|_A^2} \leq \varepsilon$ is bounded in $\mathcal{O}\left(t_A \sqrt{\kappa} \log\left(\frac{1}{\varepsilon}\right)\right)$, where $\|x\|_A^2 = \langle x, Ax \rangle$, t_A is the cost of a matrix-vector product involving A , and κ is the condition number of A [81]. In our case, κ is bounded by²³

$$(3.3) \quad \frac{q_* + 1 + d_{\max}}{q_* + \lambda_1} \leq \kappa = \frac{q_* + \lambda_n}{q_* + \lambda_1} \leq \frac{q_* + 2d_{\max}}{q_* + \lambda_1},$$

where $\lambda_1 \leq \dots \leq \lambda_n$ are the eigenvalues of L_θ and $d_{\max}$ is the maximum degree. Note that λ_1 is a measure of the quality of the optimal angular assignment (the lower λ_1 is, the better the quality of the synchronization), which is small for our synthetic connection (good synchronization due to low incoherence). In the trivial-connection case, graphs with a highly-heterogenous degree distribution, as is the case here, tend to have poor conditioning; this follows for example from a perturbation argument (see, *e.g.*, Thm.1 in [85]).

In addition to being sensitive to conditioning, CG is sensitive to graph density: each iteration has cost t_A in $\mathcal{O}(|\mathcal{E}|)$. On the other hand, we find in practice that our algorithm is much less sensitive to conditioning, and less sensitive to density. Showing this rigorously is certainly possible but would involve a lengthy exhaustion argument, depending on the comparisons between q_* , λ_1 and $d_{\max}$. Note that the concentration bound in Proposition 2.9 does not feature the condition number, which suggests that poorly-conditioned graphs do not require more Monte-Carlo replicates. In addition, setting $q \approx \bar{d}$ in the runtime bound of Proposition 2.8 gives $\mathcal{O}(|\mathcal{V}|)$ expected runtime for sampling MTSFs, which suggests lower sensitivity to graph density.

In general, we therefore expect MTSF-based solutions to perform better than CG on poorly-conditioned or high-density graphs.

Further profiling also shows that the most expensive operations during the computation of $\bar{f}$ (and $\hat{f}$) are the instantiation of the data structures we use, and the successive memory accesses to the different θ_e . These issues will likely benefit from further democratization of randomized computational schemes [57], with more efficient optimizations and architectures specialized for these types of computations. Note also that the connection model used here is only representative of a subset of applications (*e.g.* [84], [16] or [26], see also Remark 1.1).

Choice of q . In practice, the value of q_* is unknown, and the choice of the regularization parameter q is a classical model-selection issue, that goes beyond the specific case of graphs, and can be approached from different perspectives.

- It may be inferred from an application-specific perspective (this is the case in the vector-field extension problem of [71]).

²³These bounds hold because $1 + d_{\max} \leq \lambda_n \leq 2d_{\max}$, as we show in the Supplementary Material (Section 12).

- From a statistical-signal-processing perspective, it may be estimated by inspecting different criteria, such as (for instance) Akaike’s information criterion, the Bayesian information Criterion, leave-one-out cross validation, or Stein’s unbiased risk estimator (in the case of graph Tikhonov regularization for trivial connections, this last criterion is discussed in detail in [61]).

The methods mentioned in this second point require the computation of the *effective degrees of freedom* of the model [31], for different values of q . Here, this quantity takes the form

$$(3.4) \quad s(q) = \text{tr} \left(q(\mathbf{L}_\theta + q\mathbf{I})^{-1} \right).$$

In the *trivial-connection setting*, it is known that an unbiased estimator of $s(q)$ is obtained by counting the number of roots of MTSFs: $s(q) = \mathbf{E}_{\phi \sim \mathcal{D}_\mathcal{M}} (|\phi \cap \mathcal{V}|)$ [6]; in conjunction with variance-reduction techniques, this method reaches state-of-the-art performance [62]. An important corollary of our work is the extension of this strategy to *any connection*, as we detail in the Supplementary Material (Remark 9.5).

4. Randomized Angular Synchronization. We describe in this section how our novel estimators can be successfully applied to angular synchronization. Recall from Section 1.2 that, to perform angular synchronization, we aim to minimize the incoherence

$$(4.1) \quad \underset{f \in U(\mathbf{C})^n}{\text{argmin}} \langle f, \mathbf{L}_\theta f \rangle,$$

which is NP-hard in general [86]. We go over existing approaches before introducing in Section 4.1 a randomized scheme based on a spectral relaxation of Problem (4.1).

Spectral relaxation and other existing approaches. Problem (4.1) is often relaxed to the following form [75, 84]:

$$(4.2) \quad \underset{\|f\|_2^2 = n}{\text{argmin}} \langle f, \mathbf{L}_\theta f \rangle,$$

the solution of which is given by the eigenvector associated to the *smallest* eigenvalue of $\mathbf{L}_\theta$. This solution differs from the exact solution of Problem (4.1) (even though guarantees on its quality exist, e.g. [24]), and can be computed using either an inverse power method, a Rayleigh quotient iteration, or a Lanczos iteration [68]. Most theoretical studies focus on Erdős-Rényi graphs, and suggest that spectral relaxations work best in small noise regimes (as compared with other algorithms, regarding the resulting error) [10, 47, 60]. Other existing methods include (see [18] for a similar discussion):

- Semi-definite relaxations [75], providing flexible theoretical tools and performance similar to spectral relaxations, but impractical past mid-sized instances ($n \simeq 10^4$).
- The *generalized power method* proposed in [10], provably reaching the optimal solution of Problem (4.1) in the presence of (low) Gaussian noise, on complete graphs.
- Graph Neural Networks, with state-of-the-art performance for high noise [32, 35].
- Message-passing algorithms. For Gaussian noise, Approximate Message Passing has been conjectured to be statistically optimal among polynomial time algorithms [60], even at higher noise levels, but is limited to very dense graphs. Cycle-Edge Message-Passing allows exact recovery under a theoretical corruption model (with linear rate), but is more computationally-expensive than spectral relaxations [47].

- Descent techniques such as [53], with exact recovery under a (different) corruption model (see also [49]).

Note that message-passing algorithms and descent techniques (along with others we did not mention as well) can be applied to synchronization problems over more general groups, but are often not competitive on benchmarks for *angular* synchronization [32].

We will now discuss how to use our randomized estimators for eigenvector-computation.

4.1. Proposed Approach. One way to solve the spectral relaxation of Eq. (4.2) is to perform an inverse power iteration, by setting $f_0 \in \mathbf{C}^{\mathcal{V}}$ and iterating:

$$(4.3) \quad f_{r+1} = \frac{\mathbf{L}_\theta^{-1} f_r}{\|\mathbf{L}_\theta^{-1} f_r\|_2}$$

until convergence. Note that for this iteration to be well-defined, $\mathbf{L}_\theta$ needs to be invertible, *i.e.*, the angular-synchronization problem needs to be non-trivial. Depending on the topology of the graph $\mathbf{L}_\theta$ may be poorly conditioned, yielding a very difficult robust estimation of $\mathbf{L}_\theta^{-1} f_r$.

A classical workaround is to regularize the matrix $\mathbf{L}_\theta$. Noting that the matrices $\mathbf{L}_\theta^{-1}$ and its regularized version $(\mathbf{L}_\theta + q\mathbf{I})^{-1}$ (with a better condition number) share the same eigenvectors, it is a classical result that the power method applied to a matrix of the form $(\mathbf{L}_\theta + q\mathbf{I})^{-1}$ converges to the eigenvalue of $\mathbf{L}_\theta$ closest to $-q$ [68]. In our case q is positive, so that the power method applied to $(\mathbf{L}_\theta + q\mathbf{I})$ converges to the desired spectral solution of angular synchronization, and the iteration reads

$$(4.4) \quad f_{r+1} = \frac{(\mathbf{L}_\theta + q\mathbf{I})^{-1} f_r}{\|(\mathbf{L}_\theta + q\mathbf{I})^{-1} f_r\|_2} = \frac{q(\mathbf{L}_\theta + q\mathbf{I})^{-1} f_r}{\|q(\mathbf{L}_\theta + q\mathbf{I})^{-1} f_r\|_2}.$$

The convergence of this power method iteration is geometric with ratio

$$(4.5) \quad \frac{\mu_1}{\mu_2} = \frac{\lambda_2 + q}{\lambda_1 + q},$$

where $\mu_1 \leq \mu_2$ (resp. $\lambda_1 \leq \lambda_2$) denote respectively the two largest (resp. smallest) eigenvalues of $q(\mathbf{L}_\theta + q\mathbf{I})^{-1}$ (resp. $\mathbf{L}_\theta$) [68]. The choice of q is then a classical trade-off between:

- low values of q that induce a smaller $\frac{\lambda_2 + q}{\lambda_1 + q}$ ratio, favoring faster convergence of the power iteration;
- large values of q that result in a better conditioning of the system

Remark 4.1. One could instead maximize $\langle f, \mathbf{A}_\theta f \rangle$ in Eq. (4.1). This formulation is common in theoretical works, that mostly focus on (often dense) Erdős-Rényi graphs, and applying the power method to $\mathbf{A}_\theta$ is efficient on these graphs. However, it is often recommended to work with the smallest eigenvalue of $\mathbf{L}_\theta$ rather than the largest of $\mathbf{A}$ when the ratio α_1/α_2 of the top eigenvalues of $\mathbf{A}_\theta$ is close to 1. For trivial connections, such cases include: regular grids (with α_1/α_2 growing worse with the dimension), graphs with homogeneous spatial correlations (e.g., ε -graphs, nearest-neighbors-graphs), or graphs with homogeneous degree distributions and bottlenecks (poor expansion) such as in SBMs (for regular graphs, this is a classical Cheeger inequality). Our experiments in Section 14 of the Supplementary Material suggest that the

same observations can be made for non-trivial connections²⁴. It is also possible to consider a normalized Laplacian $\widetilde{\mathbf{L}}_\theta = \mathbf{D}^{-\frac{1}{2}} \mathbf{L}_\theta \mathbf{D}^{-\frac{1}{2}}$ in the spectral relaxation (4.2), which was proposed in [17, 84] and comes with similar guarantees [5].

Our approach simply consists in estimating each of the successive smoothings of Eq. (4.4) with MTSF-based estimators (we in fact recognize the term in $q(\mathbf{L}_\theta + q\mathbf{I})^{-1} f_r$ as precisely what we can estimate with our MTSF-based estimators). In the remainder of this section, we illustrate the performance of our strategy on a toy problem, and compare it for different graph topologies to a smoothing performed by CG at each step of the power iteration. Further, we also illustrate in Section 14 of the Supplementary Material that, as compared to the power iteration applied to the matrix $\mathbf{A}_\theta$, our strategy can result in very significant speed-ups on graphs for which the top spectral gap of $\mathbf{A}_\theta$ is small (as discussed in Remark 4.1).

Note that our approach is flexible enough to compute the bottom eigenvector of the normalized connection Laplacian $\widetilde{\mathbf{L}}_\theta$ as well (Remark 4.1). Indeed, our MTSF-based estimators extend to quantities of the form $f_\circ = (\mathbf{L}_\theta + q\mathbf{D})^{-1}(q\mathbf{D})g'$ (see Section 9 of the Supplementary Material), which allows to estimate $q(\widetilde{\mathbf{L}}_\theta + q\mathbf{I})^{-1}g$ by setting $g' = \mathbf{D}^{-\frac{1}{2}}g$ and noting that:

$$(4.6) \quad q(\widetilde{\mathbf{L}}_\theta + q\mathbf{I})^{-1}g = \mathbf{D}^{\frac{1}{2}}((\mathbf{L}_\theta + q\mathbf{D})^{-1}(q\mathbf{D}))g'.$$

4.2. Illustration: Intra-Class Denoising. Before delving into precise computation time comparisons with state-of-the-art methods, we first illustrate our proposed method on a toy denoising problem, inspired from an application in cryo-EM [70, 88].

We consider n copies $(I_i)_{i=1}^n$ of some image I_* that have been rotated and degraded:

$$(4.7) \quad I_i = r_i(I_*) + \varepsilon,$$

with r_i a rotation and $\varepsilon \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$ some Gaussian noise. The rotations r_i are unknown, and the goal is to recover image I_* . This is a simplified version of the intra-class denoising problem in cryo-EM, where each (2D) image corresponds to a noisy projection of a (3D) molecule observed in an unknown orientation.

In the absence of rotations r_i , a solution consists in averaging the I_i 's, trivially recovering I_* as $n \rightarrow \infty$. This strategy fails in our setting (due to the rotations), and we need to estimate the r_i 's before denoising. To do that, we first estimate angles $\theta_{i,j}$ between a subset of pair of images $\mathcal{E}$ (representing the edges of a graph) using image moments [29]²⁵, and perform angular synchronization to estimate the rotations r_i 's. We then rotate and average the images I_i 's.

We take I_* the 256×256 Shepp-Logan phantom [73], $n = 1000$, $q = 10^{-3}$, $k = 20$ iterations of the power method, and randomly choose the underlying graph $\mathcal{G} \sim ER(n, p)$ with $p = \frac{5}{n}$. We uniformly sample rotations r_i with angles in $(-\frac{\pi}{2}, \frac{\pi}{2})$ ²⁶. We display the recovered images in Figure 4.1 (solving the spectral relaxation (4.2) using both our method, with $m = 10$ MTSFs for the estimator $\bar{f}$ at each step, and an exact solver), for different noise levels $\sigma^2 \in \{1, 3, 5, 10\}$.

²⁴Further, Cramér-Rao bounds for angular synchronization suggest that angular synchronization is (statistically) difficult on these graphs [11].

²⁵Specifically, we estimate the orientation of each of the two images using the eigenvectors of the matrix of its covariant moments, and take $\theta_{i,j}$ the angle between these two sets of eigenvectors

²⁶We restrict the possible rotations so that we can use the simple image-moment-based registration, other methods could be used instead.

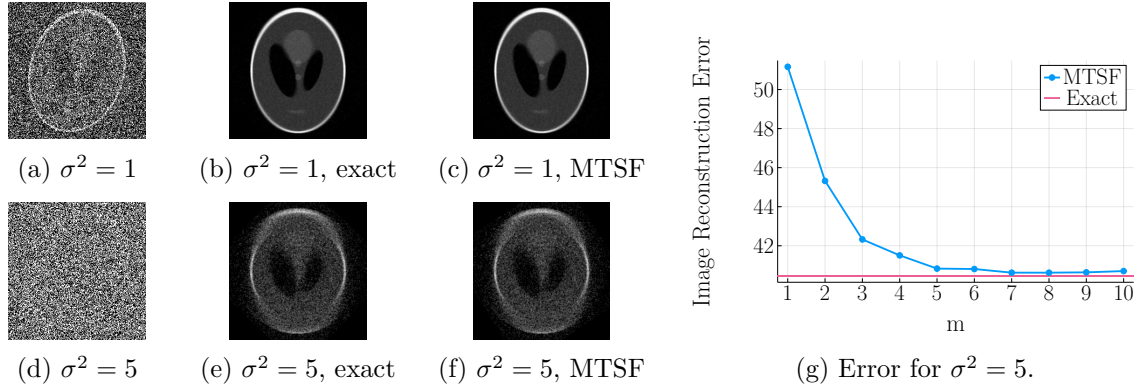


Figure 4.1: Examples of recovery from noisy images (left), with different noise levels, using both the exact solution to the spectral relaxation (4.2) and the approach from Section 4.1.

We also plot the image reconstruction error $e_I(x) = \|I_* - x\|_2$ for $\sigma^2 = 5$ with varying values of m . Here, we have no guarantee that the connection resulting from the image-moment-based estimation is weakly-incoherent, and *perform importance sampling* according to the importance distribution of Eq. (2.9)²⁷, based on the estimator $\bar{f}$.

Even though the connection may not be weakly-incoherent, the MTSF-based synchronization allows to obtain results of a quality very similar to exact synchronization on this problem, even at higher noise levels.

We propose in Section 15 of the Supplementary Material a similar application on the MNIST dataset, where we do not control the incoherence of the connection, nor the noise.

We will now investigate trade-offs between the quality of the synchronization and the runtime of our methods depending on the solver used to compute the power-method iteration in Eq. (4.4), and the topology of the graph.

4.3. Numerical Evaluation. We compare the performance of the method from Section 4.1 on synthetic graph data using four different iterative solvers: $\bar{f}$, $\hat{f}$ and a conjugate-gradient descent with and without diagonal preconditioning.

Setup. We work with the SBM and DC-SBM graph models discussed in Section 3.2. We endow each graph with a random connection generated according to the model described in Section 3, for both weakly-incoherent ($\eta = \frac{\pi}{2n}$) and incoherent connections ($\eta = \frac{\pi}{10}$), and aim to recover the vector x with $x_i = e^{i\omega_i}$. Starting from f_0 taken uniformly in $U(\mathbf{C})^n$, we perform k iterations of the power iteration of Eq. (4.4), estimating the solution with m MTSFs (resp. conjugate-gradient iterations) for each method. We use $m = 3$ for weakly-incoherent connections, and $m = 10$ for incoherent ones. For incoherent connections, we use the importance-sampling distribution from Eq. (2.9). We average the synchronization errors

$$(4.8) \quad e_s(f) = \min_{r \in U(\mathbf{C})} \frac{\|f - rx\|_2}{n}$$

²⁷Here, there is no issue with estimating af_* instead of f_* , due to the re-normalization in Eq. (4.4).

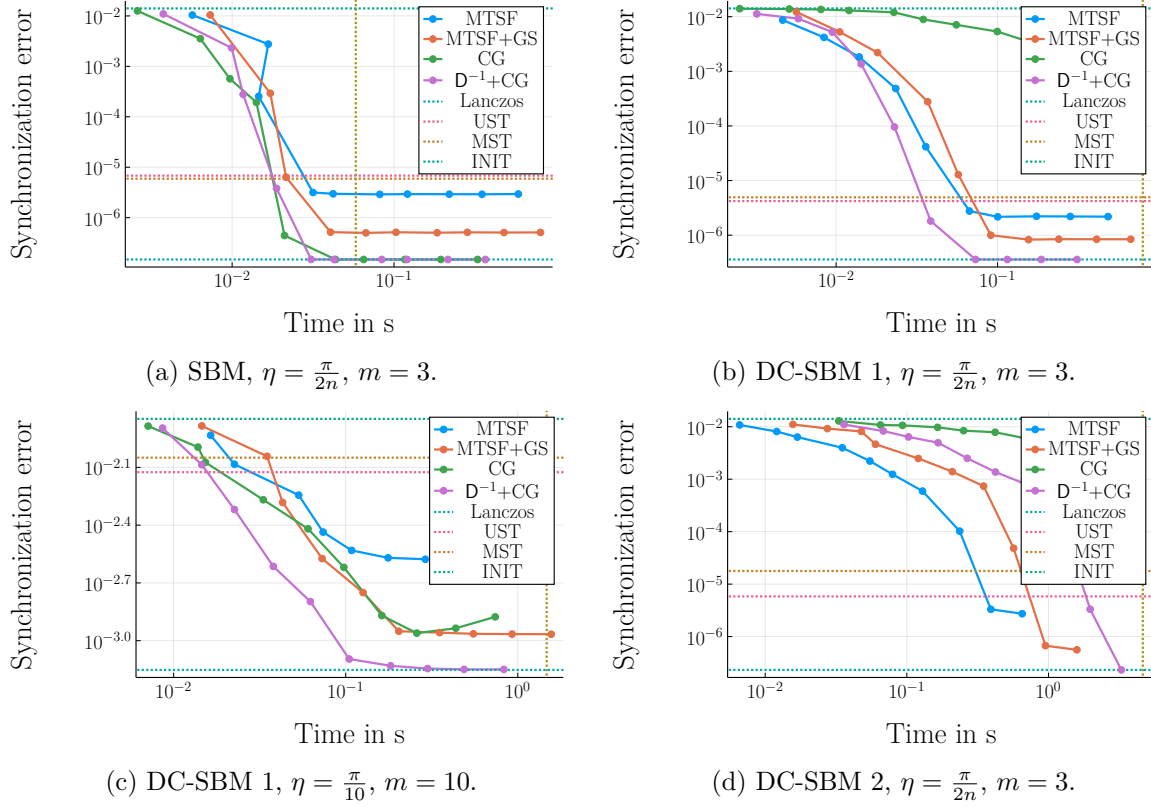


Figure 4.2: Runtime-precision trade-offs for angular synchronization. Each data point corresponds to a value of k . The vertical yellow line records the average runtime of the Lanczos-based computation, the horizontal blue line the synchronization error for the Lanczos-based computation. The horizontal red (resp. orange) line is the mean synchronization error for the UST-based (resp. MST-based) synchronization, and the horizontal teal line the error for the random initialization.

over 20 executions, and measure the mean runtimes over 100 runs. We set $q = 10^{-2} \times \bar{d}$ and take measurements for different values of k (10 logarithmically-spaced values in $\{1, \dots, 100\}$). The results are averaged over 10 realizations of the random graphs. See Figure 4.2.

We also take measurements for a Lanczos-iteration-based computation²⁸ (for the matrix L_θ), and for a naive synchronization algorithm going as follows. First, fix a root node $v \in V$, then:

- sample a spanning tree of G uniformly (a UST, using Wilson’s algorithm [82]),
- propagate the value from v to the other nodes (taking into account the offsets).

Note that this procedure does *not* depend on k or m . Finally, we also evaluate a similar strategy propagating along a *maximum spanning tree* (MST), with respect to the edge-weights $w_{i,j} = |\cos(\theta_{i,j})|$ (a strategy inspired from surface reconstruction techniques such as [33]).

²⁸We use the one implemented in <https://jutho.github.io/KrylovKit.jl/stable/man/eig/#KrylovKit.eigsolve>.

Comments. Results vary greatly depending on the graph and connection. For weakly-incoherent connections ($\eta = \frac{\pi}{2n}$), we observe the following:

- (1) Krylov-subspaces-based methods are sensitive to the conditioning : all methods perform similarly on the SBM, but not on the (less well-conditioned) DC-SBM graphs.
- (2) Higher density results in significant slow-downs for Krylov-subspaces methods, and MTSF-based iterations have a much better performance in this case (DC-SBM 2).
- (3) For weakly-inconsistent connections, the tree-based methods also achieve good synchronization quality, but this is no longer the case for incoherent connections.

If the first two factors combine (DC-SBM 2), our methods offer a “10 times” speed-up.

Remark 4.2. *We also experimented with two other estimation strategies (not shown): the spectral relaxation using the normalized Laplacian $\widetilde{\mathbf{L}}_\theta$, and a generalized-power-method-like algorithm, replacing the global normalization in Eq. (4.4) by a component-wise normalization, so that each f_k belongs to $U(\mathbf{C})^n$ (like in [10]). In our experiments, we did not observe any qualitative differences with the method described in Section 4.1.*

For spectral relaxations, our results suggest that MTSF-based solutions perform no-worse than deterministic solutions, and get comparatively better as the density of the graph increases. These techniques are mostly suitable for approximations, especially since previous studies (e.g. [60]) suggest that spectral-relaxation-based techniques mainly offer good-quality solutions at low noise-level, but are otherwise *sub-optimal*. One drawback of our strategy is that it requires to set a value for q : further refinements may include adapting our methodology to Rayleigh-quotient iterations [78], such that q no longer needs to be fixed, and that better convergence rates may be obtained.

5. Conclusion and Perspectives. We propose MTSF-based estimators for connection-aware smoothing and angular synchronization, reaching competitive performance on both problems, especially in low-precision regimes, and even *without* implementing parallelization. Our estimators perform propagation along the branches of MTSFs and, compared to standard deterministic methods, have another noteworthy advantage: they are less sensitive to both the density of the graph *and* the conditioning of the system considered (which is typically bad for graphs with broad degree distributions). Our techniques can apply to a variety of other problems: one result we did not include is the extension of the smoothing estimator to the interpolation problem (*i.e.*, extending a signal that is known only on a few nodes to all the graph), which may find applications to *e.g.* synchronization in presence of anchor nodes.

Our work opens a number of research directions. On the theoretical side: i) can the weak-inconsistency condition (Condition 2.6) be lifted without resorting to importance sampling strategies? We include a few preliminary developments in this direction in Section 16 of the Supplementary Material. ii) Can similar estimators be developed for $O(\mathbf{R}^d)$ synchronization? iii) Instead of using a fixed number m of MTSFs for the estimation (or CG iterations) of each of k iterations of the power method, how should those km MTSFs be used to achieve maximal precision? Alternatively, we could re-use the set of m MTSFs for each of the k iterations of the power method.

From a more applied perspective, many applications of angular synchronization only consider the *exact* solution of the spectral relaxation, but how much loss in precision (and gain in

speed) is actually acceptable? Our estimators can be seamlessly implemented on distributed systems (with communication complexity of the order of the number of steps in the sampling algorithm): can this be leveraged in applications? Can random decompositions such as MTSFs be useful in other inference algorithms, *e.g.*, message-passing algorithms [47, 54]?

REFERENCES

- [1] B. ALEXEEV, A. S. BANDEIRA, M. FICKUS, AND D. G. MIXON, *Phase Retrieval with Polarization*, SIAM J. Imaging Sci., 7 (2014), pp. 35–66.
- [2] M. ARIE-NACHIMSON, S. Z. KOVALSKY, I. KEMELMACHER-SHLIZERMAN, A. SINGER, AND R. BASRI, *Global motion estimation from point matches*, in 3DIMPVT, IEEE, 2012.
- [3] L. AVENA AND A. GAUDILLIÈRE, *Random spanning forests, Markov matrix spectra and well distributed points*, arXiv preprint arXiv:1310.1723, (2013).
- [4] ———, *Two applications of random spanning forests*, Journal of Theoretical Probability, (2018).
- [5] A. S. BANDEIRA, A. SINGER, AND D. A. SPIELMAN, *A Cheeger inequality for the graph connection Laplacian*, SIAM Journal on Matrix Analysis and Applications, 34 (2013), pp. 1611–1630.
- [6] S. BARTHELME, N. TREMBLAY, A. GAUDILLIÈRE, L. AVENA, AND P.-O. AMBLARD, *Estimating the inverse trace using random forests on graphs*, in GRETSI 2019, Lille, France, Aug. 2019.
- [7] D. BLACKWELL, *Conditional expectation and unbiased sequential estimation*, The Ann. of Mathematical Statistics, (1947).
- [8] B. BOLLOBÁS, *Modern Graph Theory*, vol. 184, Springer Science & Business Media, 1998.
- [9] P. BOUBOULIS, *Wirtinger’s calculus in general Hilbert spaces*, arXiv preprint arXiv:1005.5170, (2010).
- [10] N. BOUMAL, *Nonconvex Phase Synchronization*, SIAM J. Optim., 26 (2016), pp. 2355–2377.
- [11] N. BOUMAL, A. SINGER, P.-A. ABSIL, AND V. D. BLONDEL, *Cramér–Rao bounds for synchronization of rotations*, Information and Inference: A Journal of the IMA, 3 (2014), pp. 1–39.
- [12] D. BRYDGES, J. FRÖHLICH, AND E. SEILER, *On the construction of quantized gauge fields. I. General results*, Annals of Physics, 121 (1979), pp. 227–284.
- [13] É. CARTAN, *Les groupes d’holonomie des espaces généralisés*, Acta mathematica, 48 (1926), pp. 1–42.
- [14] S. CHEN, G. HUANG, G. PICCIOLI, AND L. ZDEBOROVÁ, *Planted X Y model: Thermodynamics and inference*, Physical Review E, 106 (2022), p. 054115.
- [15] F. R. CHUNG, *Spectral graph theory*, vol. 92, American Mathematical Soc., 1997.
- [16] M. CUCURINGU, *Sync-Rank: Robust Ranking, Constrained Ranking and Rank Aggregation via Eigenvector and SDP Synchronization*, IEEE Trans. Netw. Sci. Eng., 3 (2016), pp. 58–79.
- [17] M. CUCURINGU, Y. LIPMAN, AND A. SINGER, *Sensor network localization by eigenvector synchronization over the Euclidean group*, ACM Transactions on Sensor Networks (TOSN), 8 (2012), pp. 1–42.
- [18] M. CUCURINGU AND H. TYAGI, *An extension of the angular synchronization problem to the heterogeneous setting*, arXiv preprint arXiv:2012.14932, (2020).
- [19] M. DEREZINSKI AND M. W. MAHONEY, *Determinantal point processes in randomized numerical linear algebra*, Notices of the American Mathematical Society, 68 (2021), pp. 34–45.
- [20] M. DEREZINSKI AND M. K. WARMUTH, *Reverse Iterative Volume Sampling for Linear Regression*, J. Mach. Learn. Res., 19 (2018), pp. 23:1–23:39.
- [21] P. DRINEAS AND M. W. MAHONEY, *RandNLA: randomized numerical linear algebra*, Com. ACM, (2016).
- [22] M. FANUEL AND R. BARDENET, *Sparsification of the regularized magnetic Laplacian with multi-type spanning forests*, arXiv preprint arXiv:2208.14797, (2022).
- [23] M. FANUEL AND R. BARDENET, *On the Number of Steps of CyclePopping in Weakly Inconsistent $U(1)$ -Connection Graphs*. 2024.
- [24] F. FILBIR, F. KRAHMER, AND O. MELNYK, *On recovery guarantees for angular synchronization*, Journal of Fourier Analysis and Applications, 27 (2021), p. 31.
- [25] R. FORMAN, *Determinants of Laplacians on graphs*, Topology, 32 (1993), pp. 35–46.
- [26] S. FURUTANI, T. SHIBAHARA, M. AKIYAMA, K. HATO, AND M. AIDA, *Graph Signal Processing for Directed Graphs Based on the Hermitian Laplacian*, in ECML PKDD, 2019.
- [27] D. GABAY AND B. MERCIER, *A dual algorithm for the solution of nonlinear variational problems via*

- finite element approximation*, Computers & mathematics with applications, 2 (1976), pp. 17–40.
- [28] A. GIRIDHAR AND P. R. KUMAR, *Distributed Clock Synchronization over Wireless Networks: Algorithms and Analysis*, in CDC, IEEE, 2006.
- [29] R. C. GONZALEZ, *Digital image processing*, Pearson education india, 2009.
- [30] L. GULIKERS, M. LELARGE, AND L. MASSOULIÉ, *Non-backtracking spectrum of degree-corrected stochastic block models*, arXiv preprint arXiv:1609.02487, (2016).
- [31] T. HASTIE, R. TIBSHIRANI, J. H. FRIEDMAN, AND J. H. FRIEDMAN, *The elements of statistical learning: data mining, inference, and prediction*, vol. 2, Springer, 2009.
- [32] Y. HE, G. REINERT, D. WIPF, AND M. CUCURINGU, *Robust Angular Synchronization via Directed Graph Neural Networks*, arXiv preprint arXiv:2310.05842, (2023).
- [33] H. HOPPE, T. DEROSE, T. DUCHAMP, J. McDONALD, AND W. STUETZLE, *Surface Reconstruction from Unorganized Points*, in SIGGRAPH, 1992.
- [34] J. B. HOUGH, M. KRISHNAPUR, Y. PERES, AND B. VIRÁG, *Determinantal Processes and Independence*, Probability Surveys, 3 (2006), pp. 206 – 229.
- [35] N. JANCO AND T. BENDORY, *Unrolled algorithms for group synchronization*, IEEE Open Journal of Signal Processing, (2023).
- [36] H. JAQUARD, M. FANUEL, P.-O. AMBLARD, R. BARDENET, S. BARTHELMÉ, AND N. TREMBLAY, *Smoothing Complex-Valued Signals on Graphs with Monte-Carlo*, in ICASSP, 2023.
- [37] B. KARRER AND M. E. NEWMAN, *Stochastic blockmodels and community structure in networks*, Physical review E, 83 (2011), p. 016107.
- [38] A. KASSEL, *Learning about critical phenomena from scribbles and sandpiles*, ESAIM: Proceedings and Surveys, 51 (2015), pp. 60–73.
- [39] A. KASSEL AND R. KENYON, *Random curves on surfaces induced from the Laplacian determinant*, The Annals of Probability, 45 (2017), pp. 932 – 964.
- [40] A. KASSEL AND T. LÉVY, *Covariant symanzik identities*, Probability and Mathematical Physics, (2021).
- [41] R. KENYON, *Spanning forests and the vector bundle japlacian*, The Ann. of Probability, (2011).
- [42] D. P. KROESE, T. TAIMRE, AND Z. I. BOTEV, *Handbook of monte carlo methods*, 2013.
- [43] A. B. KUIJLAARS, *Convergence analysis of Krylov subspace iterations with methods from potential theory*, SIAM review, 48 (2006), pp. 3–40.
- [44] A. KULEZA AND B. TASKAR, *Determinantal Point Processes for Machine Learning*, Found. Trends Mach. Learn., 5 (2012), pp. 123–286.
- [45] R. KYNG AND Z. SONG, *A matrix chernoff bound for strongly rayleigh distributions and spectral sparsifiers from a few random spanning trees*, in FOCS, IEEE, 2018.
- [46] Y. LECUN, L. BOTTOU, Y. BENGIO, AND P. HAFFNER, *Gradient-based learning applied to document recognition*, Proceedings of the IEEE, 86 (1998), pp. 2278–2324.
- [47] G. LERMAN AND Y. SHI, *Robust Group Gynchronization via Cycle-Edge Message Passing*, Foundations of Computational Mathematics, 22 (2022), pp. 1665–1741.
- [48] J. LESKOVEC AND A. KREVL, *SNAP Datasets: Stanford Large Network Dataset Collection*, 2014.
- [49] H. LIU, X. LI, AND A. M.-C. SO, *ReSync: Riemannian Subgradient-based Robust Rotation Synchronization*, arXiv preprint arXiv:2305.15136, (2023).
- [50] P. LORENZO, S. BARBAROSSA, AND P. BANELLI, *Sampling and recovery of graph signals*, in Cooperative and Graph Signal Processing, Elsevier, 2018, pp. 261–282.
- [51] O. MACCHI, *The coincidence approach to stochastic point processes*, Adv. in Applied Probability, (1975).
- [52] P. MARTINSSON AND J. A. TROPP, *Randomized numerical linear algebra: Foundations and algorithms*, Acta Numer., 29 (2020), pp. 403–572.
- [53] T. MAUNU AND G. LERMAN, *Depth descent synchronization in $SO(d)$* , Int. journ. of comp. vision, (2023).
- [54] M. MEZARD AND A. MONTANARI, *Information, physics, and computation*, Oxford University Press, 2009.
- [55] L. A. S. MOREIRA, A. L. L. RAMOS, M. L. R. DE CAMPOS, J. A. APOLINÁRIO, AND F. G. SERRENHO, *A Graph Signal Processing Approach to Direction of Arrival Estimation*, in EUSIPCO 2019.
- [56] M. E. MULLER, *Some continuous Monte Carlo methods for the Dirichlet problem*, The Annals of Mathematical Statistics, (1956), pp. 569–589.
- [57] R. MURRAY, J. DEMMEL, M. W. MAHONEY, N. B. ERICHSON, M. MELNICHENKO, O. A. MALIK, L. GRIGORI, P. LUSZCZEK, M. DEREZIŃSKI, M. E. LOPES, ET AL., *Randomized numerical linear algebra: A perspective on the field with an eye to software*, arXiv:2302.11474, (2023).

- [58] A. B. OWEN, *Monte carlo theory, methods and examples*, 2013.
- [59] N. PARIKH, S. BOYD, ET AL., *Proximal algorithms*, Foundations and trends® in Optimization, (2014).
- [60] A. PERRY, A. S. WEIN, A. S. BANDEIRA, AND A. MOITRA, *Message-Passing Algorithms for Synchronization Problems over Compact Groups*, Com. on Pure and Applied Mathematics, (2018).
- [61] Y. Y. PILAVCI, P. AMBLARD, S. BARTHELMÉ, AND N. TREMBLAY, *Graph Tikhonov Regularization and Interpolation Via Random Spanning Forests*, IEEE Trans. Signal Inf. Process. over Networks.
- [62] Y. Y. PILAVCI, P.-O. AMBLARD, S. BARTHELME, AND N. TREMBLAY, *Variance reduction for inverse trace estimation via random spanning forests*, arXiv preprint arXiv:2206.07421, (2022).
- [63] Y. Y. PILAVCI, P.-O. AMBLARD, S. BARTHELMÉ, AND N. TREMBLAY, *Variance Reduction in Stochastic Methods for Large-Scale Regularized Least-Squares Problems*, in EUSIPCO, IEEE, 2022.
- [64] G. PUY, N. TREMBLAY, R. GRIBONVAL, AND P. VANDERGHEYNST, *Random sampling of bandlimited signals on graphs*, Applied and Computational Harmonic Analysis, 44 (2018), pp. 446–475.
- [65] T. C. RAIA, M. G. P. THOMAS, F. G. SERRENHO, AND J. A. APOLINÁRIO JR, *Gsp-based doa estimation for a multimission radar*, SBrT2020, (2020).
- [66] C. R. RAO, *Information and the accuracy attainable in the estimation of statistical parameters*, in Breakthroughs in Statistics: Foundations and basic theory, Springer, 1992, pp. 235–247.
- [67] Y. SAAD, *Iterative methods for sparse linear systems*, SIAM, 2003.
- [68] ———, *Numerical methods for large eigenvalue problems: revised edition*, SIAM, 2011.
- [69] R. SAWHNEY AND K. CRANE, *Monte Carlo geometry processing: A grid-free approach to PDE-based methods on volumetric domains*, ACM Transactions on Graphics, 39 (2020).
- [70] S. H. SCHERES, *RELION: implementation of a Bayesian approach to cryo-EM structure determination*, Journal of structural biology, 180 (2012), pp. 519–530.
- [71] SHARP, NICHOLAS AND SOLIMAN, YOUSUF AND CRANE, KEENAN, *The vector heat method*, TOG, (2019).
- [72] J. SHARPNACK, A. SINGH, AND A. RINALDO, *Sparsistency of the edge lasso over graphs*, in Artificial Intelligence and Statistics, PMLR, 2012, pp. 1028–1036.
- [73] L. A. SHEPP AND B. F. LOGAN, *The Fourier reconstruction of a head section*, IEEE Transactions on nuclear science, 21 (1974), pp. 21–43.
- [74] D. I. SHUMAN, S. K. NARANG, P. FROSSARD, A. ORTEGA, AND P. VANDERGHEYNST, *The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains*, IEEE signal processing magazine, 30 (2013), pp. 83–98.
- [75] A. SINGER, *Angular synchronization by eigenvectors and semidefinite programming*, Applied and computational harmonic analysis, 30 (2011), pp. 20–36.
- [76] A. SINGER AND H.-T. WU, *Vector diffusion maps and the connection Laplacian*, Communications on pure and applied mathematics, 65 (2012), pp. 1067–1144.
- [77] D. A. SPIELMAN AND N. SRIVASTAVA, *Graph sparsification by effective resistances*, in Proceedings of the fortieth annual ACM symposium on Theory of computing, 2008, pp. 563–568.
- [78] L. N. TREFETHEN AND D. BAU, *Numerical linear algebra*, SIAM, 2022.
- [79] N. TREMBLAY, P. GONÇALVES, AND P. BORGNAT, *Design of graph filters and filterbanks*, in Cooperative and Graph Signal Processing, Elsevier, 2018, pp. 299–324.
- [80] J. A. TROPP, *User-friendly tail bounds for sums of random matrices*, Founds. of comput. maths., (2012).
- [81] N. K. VISHNOI ET AL., $Lx = b$, Foundations and Trends® in Theoretical Computer Science, (2013).
- [82] D. B. WILSON, *Generating Random Spanning Trees More Quickly than the Cover Time*, in STOC, 1996.
- [83] S. X. YU, *Angular embedding: From jarring intensity differences to perceived luminance*, in CVPR, 2009.
- [84] ———, *Angular Embedding: A Robust Quadratic Criterion*, Trans. Pattern Anal. Mach. Intell., (2012).
- [85] C. ZHAN, G. CHEN, AND L. F. YEUNG, *On the distributions of laplacian eigenvalues versus node degrees in complex networks*, Physica A: Statistical Mechanics and its Applications, 389 (2010), pp. 1779–1788.
- [86] S. ZHANG AND Y. HUANG, *Complex quadratic optimization and semidefinite programming*, SIAM Journal on Optimization, 16 (2006), pp. 871–890.
- [87] X. ZHANG, Y. HE, N. BRUGNONE, M. PERLMUTTER, AND M. J. HIRN, *MagNet: A Neural Network for Directed Graphs*, in NeurIPS, 2021.
- [88] Z. ZHAO AND A. SINGER, *Rotationally invariant image representation for viewing direction classification in cryo-EM*, Journal of structural biology, 186 (2014), pp. 153–166.

Supplementary Material

We work with a weighted graph $\mathcal{G}$ in all the following proofs, with $w_e \in \mathbf{R}_+$ the weight of edge e . We still denote by $\mathbf{L}_\theta$ the resulting (weighted) connection Laplacian, with off-diagonal entries $(\mathbf{L}_\theta)_{i,j} = -w_{i,j}e^{i\theta(j,i)}$ if $\{i,j\} \in \mathcal{E}$ ($(\mathbf{L}_\theta)_{i,j} = w_{(i,j)} = 0$ otherwise), and diagonal $(\mathbf{L}_\theta)_{i,i} = d_i = \sum_j w_{i,j}$ the weighted degree of node i .

6. Proof of Proposition 2.1. Denote by $C : \mathbf{C}^\mathcal{V} \rightarrow \mathbf{R}$ the cost function

$$(6.1) \quad C(f) = q\|f - g\|_2^2 + \langle f, \mathbf{L}_\theta f \rangle.$$

We use a standard argument from **CR**-calculus, and seek the zeros of the Fréchet Wirtinger derivatives of C (see *e.g.* [9]). Let us first compute these derivatives: for $f, h \in \mathbf{C}^\mathcal{V}$, we have

$$(6.2) \quad C(f+h) = q\langle (f+h) - g, (f+h) - g \rangle + \langle (f+h), \mathbf{L}_\theta(f+h) \rangle,$$

$$(6.3) \quad = C(f) + D_f^W C(h) + D_f^{W*} C(h) + (q\langle h, h \rangle + \langle h, \mathbf{L}_\theta h \rangle),$$

where $(h \mapsto q\langle h, h \rangle + \langle h, \mathbf{L}_\theta h \rangle) \in o(h)$, and the associated (conjugate) Fréchet-Wirtinger derivatives of C (at f) $D_f^W C$ and $D_f^{W*} C : \mathbf{C}^\mathcal{V} \rightarrow \mathbf{C}$ are given by:

$$(6.4) \quad D_f^W C(h) = \langle q(f - g) + \mathbf{L}_\theta h, h \rangle$$

$$(6.5) \quad D_f^{W*} C(h) = \langle h, q(f - g) + \mathbf{L}_\theta \rangle.$$

The result follows from the fact that $D_f^W C = 0$ (resp. $D_f^{W*} C = 0$) identically if and only if $f = q(\mathbf{L}_\theta + q\mathbf{I})^{-1}g$.

7. A Generalization of Property (P2). Define $\Delta_\theta = \mathbf{D}^{-1}\mathbf{L}_\theta$ the weighted connection-aware random walk Laplacian Δ_θ . Proposition 7.1 generalizes property (P2).

Proposition 7.1. For all $l \geq 1$, we have

$$(7.1) \quad (\mathbf{I} - \Delta_\theta)_{i,j}^l = \sum_{\substack{p \in P_i^j \\ l(p)=l}} \left(\prod_{0 \leq k < l} \frac{w_{(u_k, u_{k+1})}}{d_{u_k}} \right) \psi_{p^*},$$

with $l(p)$ the length of a path $p = ((u_0, u_1), (u_1, u_2), \dots, (u_{l(p)-1}, u_{l(p)}))$.

For each path p , the product in the r.h.s. of Equation (7.1) is the probability of observing p when performing a random walk from i to j on G . For the trivial connection this expression is equal to $(\mathbf{D}^{-1}\mathbf{A})_{i,j}^l$, the probability of a path going from i to j in l steps, but Proposition 7.1 more broadly captures *random propagations* along the corresponding sampled paths.

Observations similar to Proposition 7.1 have been laid out in a few works, such as [40] for continuous time random walk, and in a number of other situations, where the connection often stems from theoretical physics considerations [12].

Proof. The result is obvious for $l = 1$. We proceed by induction and assume the result true for some $l \in \mathbf{N}$. Then, recalling that $(\mathbf{I} - \Delta_\theta)_{v,j} = \frac{w_{v,j}}{d_v}$, we have:

$$(7.2) \quad (\mathbf{I} - \Delta_\theta)_{i,j}^{l+1} = \left((\mathbf{I} - \Delta_\theta)^l (\mathbf{I} - \Delta_\theta) \right)_{i,j}$$

$$(7.3) \quad = \sum_{v \in \mathcal{V}} \sum_{\substack{p \in P_i^v \\ l(p)=l}} \left(\prod_{0 \leq k < l} \frac{w(u_k, u_{k+1})}{d_{u_k}} \right) \frac{w_{v,j}}{d_v} \psi_{p^*} \psi_{(v,j)^*}$$

$$(7.4) \quad = \sum_{\substack{p \in P_i^j \\ l(p)=l+1}} \left(\prod_{0 \leq k < l+1} \frac{w(u_k, u_{k+1})}{d_{u_k}} \right) \psi_{p^*}. \quad \blacksquare$$

Remark 7.2. For $l = 0$, we have $(\mathbf{I} - \Delta_\theta)^l = \mathbf{I}$.

8. Proof of Proposition 2.2. We will show an extension of Proposition 2.2 to heterogeneous values of q . That is, we associate to each node i some non-negative weight $q_i \in \mathbf{R}$ (at least one of them needs to be *positive*), that we record in the diagonal matrix $\mathbf{Q}$ with $\mathbf{Q}_{i,i} = q_i$. These q_i 's can be interpreted as edge weights in the extended graph $\mathcal{G}^\Gamma$, and we extend the definition of the measure ν_i over P_i^Γ to account for these weights, so that

$$(8.1) \quad \mathbf{P}_{\nu_i}(p) = \mathbf{1}_{\{i\}}(u_0) \mathbf{1}_{\{\Gamma\}}(u_l) \prod_{0 \leq k < l} \left(\mathbf{1}_{\mathcal{V} \setminus \{\Gamma\}}(u_k) \frac{w(u_k, u_{k+1})}{d_{u_k} + q_{u_k}} \right)$$

for a path $p = ((u_0, u_1), \dots, (u_{l-1}, u_l))$ of length l , and $\mathbf{1}_S(x)$ the indicator that $x \in S$. We show the following Feynman-Kac formula, on weighted graphs and for heterogeneous q_i 's.

Proposition 8.1. For j the last node reached before absorption of a path p , we have

$$(8.2) \quad \left((\mathbf{L}_\theta + \mathbf{Q})^{-1} \mathbf{Q} g \right) (i) = \mathbf{E}_{p \sim \nu_i} \left(\psi_{p_\Gamma^*}(g_j) \right).$$

Note that the l.h.s. is a more general smoothing operation than the solution to the Tikhonov problem, that can be leveraged in the presence of heteroscedastic noise (*e.g.* for the problem considered in Section 3.2 with different values of σ^2 for each node).

Proof. Consider the connection-aware random-walk Laplacian $\Delta_\theta^{\mathbf{Q}}$ associated to the extended graph $\mathcal{G}^\Gamma$. Its restriction to the rows and columns indexed by $\mathcal{V}$ reads $(\Delta_\theta^{\mathbf{Q}})_{\mathcal{V}} = (\mathbf{D} + \mathbf{Q})^{-1} (\mathbf{L}_\theta + \mathbf{Q})$. It is clearly invertible, and all its eigenvalues lie in $(0, 2)$ (for instance by the Gershgorin circle theorem), hence the eigenvalues of $(\mathbf{I} - (\Delta_\theta^{\mathbf{Q}})_{\mathcal{V}})$ are in $(-1, 1)$, ensuring the convergence of the right-hand-side expression in:

$$(8.3) \quad \left((\Delta_\theta^{\mathbf{Q}})^{-1} \right)_{i,j} = \left[\sum_{l \geq 0} (\mathbf{I} - (\Delta_\theta^{\mathbf{Q}})_{\mathcal{V}})^l \right]_{i,j}$$

$$(8.4) \quad = \sum_{p \in P_i^j} \left(\prod_{0 \leq k < l(p)} \frac{w(u_k, u_{k+1})}{d_{u_k} + q_{u_k}} \right) \psi_{p^*},$$

where Equation (8.4) follows from applying Proposition 7.1 to $\mathcal{G}^\Gamma$.

The result follows by right multiplication with $(D + Q)^{-1}Q$, where $((D + Q)^{-1}Q)_{j,j} = \frac{q_j}{d_{u_{l-1}} + q_j}$ accounts for the probability of the last transition from $u_{l-1} = j$ to Γ . $\blacksquare$

9. Proof of Theorem 2.4. Before presenting the proof of our main Theorem, we state it for weighted graphs with heterogeneous q_i 's.

Statement in the presence of weights. Consider the distribution $\mathcal{D}_{\mathcal{M}}$ over $\mathcal{M}(\mathcal{G})$ such that

$$(9.1) \quad \mathbf{P}_{\mathcal{D}_{\mathcal{M}}}(\phi) \propto \prod_{r \in \phi \cap \mathcal{V}} q_r \prod_{e \in \phi \cap \mathcal{E}} w_e \prod_{C \in \mathcal{C}(\phi)} ((2 - 2 \cos(\theta_C)),$$

and the estimator $\tilde{f}(i, \phi, g) = \psi_{r_\phi(i) \xrightarrow{\phi} i}(g(r_\phi(i)))$ propagating the value from $r_\phi(i)$ to i if i belongs to a rooted tree, and $\tilde{f}(i, \phi, g) = 0$ if i lies in a unicycle.

Theorem 9.1. Letting $\|f\|_{\mathcal{Q}}^2 = \sum_{i \in \mathcal{V}} q_i |f(i)|^2$, we have both:

$$(9.2) \quad ((L_\theta + Q)^{-1}Qg)(i) = \mathbf{E}_{\mathcal{D}_{\mathcal{M}}}(\tilde{f}(i, \phi, g)),$$

$$(9.3) \quad \mathbf{E}_{\phi \sim \mathcal{V}} (\|\tilde{f}(\phi, g) - f_*\|_{\mathcal{Q}}^2) = \|g\|_{\mathcal{Q}}^2 - \|f_*\|_{\mathcal{Q}}^2.$$

We now proceed with the proof.

Determinantal Point Processes. The first step in our proof consists in re-stating the definition of $\mathcal{D}_{\mathcal{M}}$ as a Determinantal Point Process (DPP), which will provide us with powerful tools for reasoning about MTSFs. A (discrete) DPP over a finite set $\mathcal{X}$ associates a probability to each subset $X \subseteq \mathcal{X}$, and is defined by its marginal probabilities. It is parametrized by an Hermitian matrix $K \in M_{|\mathcal{X}|}(\mathbf{C})$, whose eigenvalues must all lie in $[0, 1]$, known as the *marginal kernel* of the DPP, denoted $\text{DPP}(K)$. Precisely, we say that $X \sim \text{DPP}(K)$ if

$$(9.4) \quad \mathbf{P}_{\text{DPP}(K)}(A \subseteq X) = \det(K)_{A,A}$$

for all $A \subseteq \mathcal{X}$, where $\det(K)_{R,C}$ is the minor of K restricted to the rows and columns indexed respectively by R and C . We write $\det(K)_{\cdot, C}$ (resp. $\det(K)_{R, \cdot}$) in case $R = \mathcal{X}$ (resp. $C = \mathcal{X}$).

Marginal kernel of $\mathcal{D}_{\mathcal{M}}$. Let us now describe the marginal kernel K that we will associate to distribution $\mathcal{D}_{\mathcal{M}}$. To this end, we consider a *twisted discrete differential* operator $\nabla : \mathbf{C}^{\mathcal{V}} \rightarrow \mathbf{C}^{\mathcal{E}}$, mapping complex values defined on the complex planes $\mathbf{C}_v$ (associated to the nodes of $\mathcal{G}$) to copies of the complex planes $\mathbf{C}_e$ associated to the *edges* of $\mathcal{G}$. Its expression relies on a splitting of the connection maps $\psi_e : \mathbf{C}_{s_e} \xrightarrow{\psi_{s_e, e}} \mathbf{C}_e \xrightarrow{\psi_{e, t_e}} \mathbf{C}_{t_e}$ such that $\psi_e = \psi_{e, t_e} \circ \psi_{s_e, e}$ (this is always possible, e.g. $\psi_{e, t_e} = \text{id}_{\mathbf{C}_e, \mathbf{C}_{t_e}}$ and $\psi_{s_e, e}(z) = e^{i\theta_e} \cdot z$), and reads [41]:

$$(9.5) \quad (\nabla f)(e) = \sqrt{w_e} \psi_{t_e, e}(f(t_e)) - \sqrt{w_e} \psi_{s_e, e}(f(s_e)).$$

Expliciting the entries of the matrix of ∇ , we have:

$$(9.6) \quad \nabla_{e,v} = \begin{cases} -\sqrt{w_e} \psi_{s_e, e} & \text{if } v = s_e \\ \sqrt{w_e} \psi_{e, t_e} & \text{if } v = t_e \\ 0 & \text{otherwise.} \end{cases}$$

970 We finally take $\mathcal{X} = \mathcal{V} \cup \mathcal{E}$ and define the marginal kernel K :

$$971 \quad (9.7) \quad K = \nabla_{\mathbf{Q}}(\mathbf{L}_{\theta} + \mathbf{Q})^{-1}\nabla_{\mathbf{Q}}^*, \text{ where } \nabla_{\mathbf{Q}} = \begin{bmatrix} \nabla \\ \sqrt{\mathbf{Q}} \end{bmatrix}.$$

972

973 **Remark 9.2.** For trivial connections, ∇ is the edge-vertex incidence matrix of $\mathcal{G}$, and we
974 have $\mathbf{L} = \nabla^t \nabla$. In a similar manner, one shows that $\mathbf{L}_{\theta} = \nabla^* \nabla$.

975 We will show that $\mathcal{D}_{\mathcal{M}}$ is a DPP with kernel K . First, notice that

$$976 \quad (9.8) \quad \mathbf{L}_{\theta} + \mathbf{Q} = \nabla_{\mathbf{Q}}^* \nabla_{\mathbf{Q}},$$

977 so that K is a projection operator (*i.e.* $K^2 = K$), and all its eigenvalues are in $\{0, 1\}$. DPPs
978 associated to such kernels are known as *projection DPPs*, and can be defined without resorting
979 to marginal probabilities, with $\mathbf{P}_{\text{DPP}(K)}(X) = \det(K)_{X,X}$ when $|X| = \text{rk}(K)$ the rank of K
980 ($\text{rk}(K) = |\mathcal{V}|$ in our case), and $\mathbf{P}_{\text{DPP}(K)}(X) = 0$ otherwise [34].

981 This will allow to show that the samples of $\text{DPP}(K)$ are MTSFs, distributed according to $\mathcal{D}_{\mathcal{M}}$
982 (we generalize here an argument from [41]). Let $\phi \subseteq \mathcal{V} \cup \mathcal{E}$ with $|\phi| = |\mathcal{V}|$, and remark that:

$$983 \quad (9.9) \quad \det(K)_{\phi,\phi} = \frac{\det(\nabla_{\mathbf{Q}}^* \nabla_{\mathbf{Q}})_{\phi,\phi}}{\det(\mathbf{L}_{\theta} + \mathbf{Q})}.$$

984 The next step then consists in expliciting $\det(\nabla_{\mathbf{Q}}^* \nabla_{\mathbf{Q}})_{\phi,\phi} = \det(\nabla_{\mathbf{Q}})_{\phi,:} \det(\nabla_{\mathbf{Q}}^*)_{:, \phi}$ depending
985 on ϕ : we start by inspecting $\det(\nabla_{\mathbf{Q}}^*)_{:, \phi}$ (similar arguments apply to $\det(\nabla_{\mathbf{Q}})_{\phi,:}$). In this case,
986 ϕ indexes the columns of $\nabla_{\mathbf{Q}}$, with two columns linearly independent if they do not belong to
987 the same component c_{ϕ} . If c_{ϕ} spans m nodes, $\det(\nabla_{\mathbf{Q}}^*)_{:, c_{\phi}}$ can only be non-zero if $|c_{\phi}| = m$, so
988 that c_{ϕ} must be either a unicycle or a rooted tree. We can then compute $\det(\nabla_{\mathbf{Q}}^* \nabla_{\mathbf{Q}})_{c_{\phi}, c_{\phi}}$
989 from $\det(\nabla_{\mathbf{Q}}^*)_{:, \phi}$ and $\det(\nabla_{\mathbf{Q}})_{\phi,:}$ explicitly in those cases.

Lemma 9.3.

$$990 \quad (9.10) \quad \det(\nabla_{\mathbf{Q}}^* \nabla_{\mathbf{Q}})_{c_{\phi}, c_{\phi}} = \begin{cases} (2 - 2 \cos(\theta_C)) \prod_{e \in c_{\phi}} w_e & \text{if } c_{\phi} \text{ is a unicycle with cycle } C \\ q_r \prod_{e \in c_{\phi} \cap \mathcal{E}} w_e & \text{if } c_{\phi} \text{ is a tree rooted in } r \end{cases}$$

991

992 **Proof.** Suppose first that c_{ϕ} is a unicycle (this is the case treated in [41]). We fix an
993 orientation of the edges of c_{ϕ} such that all edges are oriented *towards* the cycle C , and that
994 the edges belonging to C form a directed cycle (there are only two such orientations, we choose
995 one arbitrarily). The only non-zero permutations in the determinant

$$996 \quad (9.11) \quad \det(\Delta_{\mathbf{Q}}^*)_{:, c_{\phi}} = \sum_{\sigma \in \mathcal{S}_m} \prod_{1 \leq i \leq m} \text{sgn}(\sigma) (\Delta_{\mathbf{Q}}^*)_{i, \sigma(i)},$$

997 where $\text{sgn}(\sigma)$ denotes the signature of permutation (bijection from nodes to edges) σ , corre-
998 spond to these orientations, and map vertices s_e to edges e ($\sigma(s_e) = e$). These two bijections,

denoted σ_C and σ_{C^*} , differ only on nodes adjacent to edges in C , and correspond to the two possible orientations of the cycle. We then obtain:

$$(9.12) \quad \det(\Delta_{\mathbf{Q}}^*)_{:,c_\phi} = (-1)^m \text{sgn}(\sigma_C) \prod_{e \in c_\phi \setminus C} \sqrt{w_e} \psi_{s_e, e} \left(\prod_{e' \in C} \sqrt{w_{e'}} \psi_{s_{e'}, e'} - \prod_{e' \in C} \sqrt{w_{e'}} \psi_{t_{e'}, e'} \right).$$

Performing a similar computation for $\det(\Delta_{\mathbf{Q}})_{c_\phi, :}$ and multiplying the two resulting expressions then allows to show that (this computation also appears in [41]):

$$(9.13) \quad \det(\nabla_{\mathbf{Q}}^* \nabla_{\mathbf{Q}})_{c_\phi, c_\phi} = (2 - 2 \cos(\theta_C)) \prod_{e \in c_\phi} w_e,$$

where we used $\psi_e = \psi_{e, t_e} \circ \psi_{s_e, e}$ and $\psi_C + \psi_{C^*} = 2 \cos(\theta_C)$.

Similarly, for c_ϕ a rooted tree with root $r \in c_\phi \cap \mathcal{V}$, we have

$$(9.14) \quad \det(\Delta_{\mathbf{Q}}^*)_{:,c_\phi} = \sqrt{q_r} \times (-1)^m \text{sgn}(\sigma_C) \prod_{e \in (c_\phi \cap \mathcal{V}) \setminus C} \sqrt{w_e} \psi_{s_e, e},$$

which results in

$$(9.15) \quad \det(\nabla_{\mathbf{Q}})_{c_\phi, c_\phi} = q_r \prod_{e \in c_\phi \cap \mathcal{E}} w_e. \quad \blacksquare$$

As a corollary, we obtain

$$(9.16) \quad \mathbf{P}_{\text{DPP}(K)}(\phi) = \frac{\prod_{r \in \phi \cap \mathcal{V}} q_r \prod_{e \in \phi \cap \mathcal{E}} w_e \prod_{C \in \mathcal{C}(\phi)} ((2 - 2 \cos(\theta_C)))}{\det(\mathbf{L}_\theta + \mathbf{Q})},$$

which is exactly $\mathbf{P}_{\mathcal{D}, \mathcal{M}}(\phi)$.

Unbiased estimator. It remains to show that $\tilde{f}(i, \phi, g)$ is an unbiased estimator of the desired quantity $((\mathbf{L}_\theta + \mathbf{Q})^{-1} \mathbf{Q}g)(i)$. Our argument is inspired from that of [61], and we will require another determinantal tool.

Proposition 9.4 (Cauchy-Binet formula). *For two matrices $A \in M_{m,n}(\mathbf{C})$ and $B \in M_{n,m}(\mathbf{C})$ with $m < n$, we have:*

$$(9.17) \quad \det(AB) = \sum_T \det(A)_{:,T} \det(B)_{T,:},$$

where T ranges over all subsets of $\{1, \dots, n\}$ of size m .

We begin by rewriting $(\mathbf{L}_\theta + \mathbf{Q})_{i,j}^{-1}$ as an expectation, starting from Cramer's rule

$$(9.18) \quad (\mathbf{L}_\theta + \mathbf{Q})_{i,j}^{-1} = (-1)^{i+j} \frac{\det(\mathbf{L}_\theta + \mathbf{Q})_{\mathcal{V} \setminus \{j\}, \mathcal{V} \setminus \{i\}}}{\det(\mathbf{L}_\theta + \mathbf{Q})}.$$

We can rewrite the numerator using the Cauchy-Binet formula:

$$(9.19) \quad \det(\mathbf{L}_\theta + \mathbf{Q})_{\mathcal{V} \setminus \{j\}, \mathcal{V} \setminus \{i\}} = \sum_{\substack{\phi \subseteq \mathcal{V} \cup \mathcal{E} \\ |\phi| = |\mathcal{V}| - 1}} (-1)^{i+j} \det\left(\left[(\nabla_{\mathbf{Q}}^*)_{:, \phi} \delta_j\right]\right) \det\left(\left[\begin{array}{c} (\nabla_{\mathbf{Q}})_{\phi, :} \\ \delta_i \end{array}\right]\right),$$

where δ_i is the i -th vector in the usual basis of $\mathbf{R}^{\mathcal{V}}$. An argument analogous to Lemma 9.3 then shows that the product of determinants

$$(9.20) \quad \det \left(\left[(\nabla_{\mathbf{Q}}^*)_{:, \phi} \delta_j \right] \right) \det \left(\begin{bmatrix} (\nabla_{\mathbf{Q}})_{\phi, :} \\ \delta_i \end{bmatrix} \right)$$

can only be non-zero if ϕ contains a tree $T_i^j \subseteq \mathcal{E}$ spanning both i and j (with no root), with contribution $\psi_{j \xrightarrow{\phi} i} \prod_{e \in T_i^j} w_e$ to the product, and if *all* the other components are rooted trees or unicycles, with associated contributions described in Lemma 9.3. We thus obtain

$$(9.21) \quad (\mathbf{L}_{\theta} + \mathbf{Q})_{i,j}^{-1} = \frac{1}{q_j} \sum_{\phi \in \mathcal{M}(\mathcal{G})} \mathbf{P}_{\phi \sim \mathcal{D}_{\mathcal{M}}}(\phi) \mathbf{1}_{c_{\phi}(j)}(i) \psi_{j \xrightarrow{\phi} i}$$

$$(9.22) \quad = \frac{1}{q_j} \mathbf{E}_{\phi \sim \mathcal{D}_{\mathcal{M}}} \left(\mathbf{1}_{c_{\phi}(j)}(i) \psi_{j \xrightarrow{\phi} i} \right),$$

where $\mathbf{1}_{c_{\phi}(j)}(i)$ is the indicator that i belongs to the set of nodes $c_{\phi}(j)$ spanned by the tree rooted in r . Finally, we have

$$(9.23) \quad \left((\mathbf{L}_{\theta} + \mathbf{Q})^{-1} \mathbf{Q} g \right) (i) = \langle \delta_i, (\mathbf{L}_{\theta} + \mathbf{Q})^{-1} \mathbf{Q} g \rangle$$

$$(9.24) \quad = \sum_{j \in \mathcal{V}} q_j (\mathbf{L}_{\theta} + \mathbf{Q})_{i,j}^{-1} g(j)$$

$$(9.25) \quad = \sum_{j \in \mathcal{V}} \mathbf{E}_{\phi \sim \mathcal{D}_{\mathcal{M}}} \left(\psi_{j \xrightarrow{\phi} i} (g(j)) \mathbf{1}_{c_{\phi}(j)}(i) \right)$$

$$(9.26) \quad = \mathbf{E}_{\phi \sim \mathcal{D}_{\mathcal{M}}} (\tilde{f}(i, \phi, g)).$$

Variance computation. The expression of the variance relies on the observation that $\tilde{f}(\phi, g)$ is *linear in g* , so that we can write $\tilde{\mathbf{S}}_{\phi} g = \tilde{f}(\phi, g)$ for some linear operator/matrix $\tilde{\mathbf{S}}_{\phi}$, and otherwise consists in a simple computation. Note that we have $\mathbf{E}_{\phi \sim \mathcal{D}_{\mathcal{M}}} (\tilde{\mathbf{S}}_{\phi} g) = f_*$ and

$$(9.27) \quad \mathbf{E}_{\phi \sim \mathcal{D}_{\mathcal{M}}} (\tilde{\mathbf{S}}_{\phi}) = (\mathbf{L}_{\theta} + \mathbf{Q})^{-1} \mathbf{Q}.$$

First, notice that $\tilde{\mathbf{S}}_{\phi}^* \mathbf{Q} \tilde{\mathbf{S}}_{\phi} = \mathbf{Q}$. We then have

$$(9.28) \quad \mathbf{E}_{\phi \sim \mathcal{D}_{\mathcal{M}}} \left(\|\tilde{S}(\phi, g) - f_*\|_{\mathbf{Q}}^2 \right) = \sum_{i \in \mathcal{V}} q_i \left(\mathbf{E} \left(\left| (\tilde{S}_{\phi} g)(i) \right|^2 \right) - \left| \mathbf{E} \left((\tilde{S}_{\phi} g)(i) \right) \right|^2 \right)$$

$$(9.29) \quad = \left(\sum_{i \in \mathcal{V}} \langle g, \mathbf{E}_{\phi \sim \mathcal{D}_{\mathcal{M}}} (\tilde{\mathbf{S}}_{\phi}^* (q_i \delta_i \delta_i^*) \tilde{\mathbf{S}}_{\phi}) g \rangle \right) - \|f_*\|_{\mathbf{Q}}^2$$

$$(9.30) \quad = \|g\|_{\mathbf{Q}}^2 - \|f_*\|_{\mathbf{Q}}^2,$$

where δ_i is usual basis vector of $\mathbf{C}^{\mathcal{V}}$ with non-zero entry $\delta_i(i) = 1$, Equation (9.29) follows from the linearity of the expectation, and Equation (9.30) from the previous observation that $\tilde{\mathbf{S}}_{\phi}^* \mathbf{Q} \tilde{\mathbf{S}}_{\phi} = \mathbf{Q}$.

Remark 9.5. We claimed in Section 3.2 that the quantity $s(q)$ in Equation (3.4) is given, in expectation, by the number of roots of MTSFs sampled according to $\mathcal{D}_{\mathcal{M}}$. This fact hinges on two well-known properties of determinantal point processes (see, e.g., [44]):

1. if X follows a DPP with over $\mathcal{X}$ with kernel K , and if $S \subseteq \mathcal{X}$, then $X \cap S$ also follows a DPP, with kernel $K_S = K_{S,S}$;
2. the cardinality $|X|$ of a DPP-sample is in general a random quantity, and its expectation is given by

$$(9.31) \quad \mathbf{E}_{X \sim \text{DPP}(K)}(|X|) = \text{tr}(K).$$

In particular, the kernel $K_{\mathcal{V}}$ obtained by restricting the kernel K of $\mathcal{D}_{\mathcal{M}}$ (Equation (9.7)) describes a distribution over the nodes $\mathcal{V}$, and reads

$$(9.32) \quad K_{\mathcal{V}} = q(\mathbf{L}_{\theta} + q\mathbf{I})^{-1}$$

or, more generally for heterogeneous q_i 's:

$$(9.33) \quad K_{\mathcal{V}} = \sqrt{\mathbf{Q}}(\mathbf{L}_{\theta} + q\mathbf{I})^{-1}\sqrt{\mathbf{Q}}.$$

The claim follows from Equation (9.31):

$$(9.34) \quad \mathbf{E}_{\phi \sim \mathcal{V}}(|\phi \cap \mathcal{V}|) = \text{tr} \left(\sqrt{\mathbf{Q}}(\mathbf{L}_{\theta} + q\mathbf{I})^{-1}\sqrt{\mathbf{Q}} \right),$$

which translates to an efficient estimator for the effective degrees of freedom of the connection-aware smoothing problem.

10. Complexity Analysis. We will describe the complexity of different implementations of the MTSF-sampling algorithm. We recall the MTSF-sampling procedure in Algorithm 10.1. We consider here the generalization to weighted graphs with heterogeneous q_i 's, and use a generalized `random_successor(u)` that outputs either Γ with probability $\frac{q_u}{d_u + q_u}$, or some node v with probability $\frac{w_{u,v}}{d_u + q_u}$. This generalized algorithms provably samples MTSFs from the distribution $\mathcal{D}_{\mathcal{M}}$, with the argument from [22] still applying.

10.1. Number of Steps. Let us re-state the upper bound on the expected number of steps of the random walk in the sampling Algorithm 2.3. Denote by T_{ϕ} the number of random neighbors sampled to build $\phi \in \mathcal{M}(\mathcal{G})$ during an execution of Algorithm 10.1. Then, we have:

Proposition 10.1.

$$(10.1) \quad \mathbf{E}_{\phi \sim \mathcal{D}_{\mathcal{M}}}(T_{\phi}) \leq \text{tr} \left((\mathbf{L} + \mathbf{Q})^{-1}(\mathbf{D} + \mathbf{Q}) \right),$$

where we abuse notation and write $\phi \sim \mathcal{D}_{\mathcal{M}}$ for ϕ sampled using Algorithm 10.1. This bound is easily obtained by considering the special case of the trivial connection with $\psi_e = \text{id}_{\mathbf{C}_{s_e}, \mathbf{C}_{t_e}}$ for all $e \in \vec{\mathcal{E}}$, in which case Algorithm 10.1 samples spanning forests of $\mathcal{G}$ from the distribution mentioned in Remark 2.5, and the expected number of steps is known to be [61]:

$$(10.2) \quad \text{tr} \left((\mathbf{L} + \mathbf{Q})^{-1}(\mathbf{D} + \mathbf{Q}) \right).$$

Algorithm 10.1 MTSF sampling algorithm [22].

```

1:  $\phi \leftarrow \emptyset$ 
2: while  $\phi$  not spanning do
3:   Let  $i \in \mathcal{V}$  be the first node in the queue not spanned by  $\phi$ 
4:    $u \leftarrow i$  ▷  $u$  is the current node of the random walk
5:    $p \leftarrow \epsilon$  ▷  $\epsilon$  the empty path
6:   while ( $u \neq \Gamma$ ) and ( $p$  does not intersect  $\phi$  or contain a cycle) do
7:      $u' \leftarrow \text{random\_successor}(u)$  ▷ Move to next node
8:     if  $u' \neq \Gamma$  then
9:        $e \leftarrow (u, u'), p \leftarrow pe$  ▷ Add  $e$  to the path  $p$ 
10:    end if
11:    if  $p$  contains a cycle  $C$  then
12:      Remove this cycle from  $p$  with probability  $\cos(\theta_C)$ 
13:    end if
14:     $u_{\text{old}} \leftarrow u, u \leftarrow u'$  ▷  $u_{\text{old}}$  the previous node
15:  end while
16:  if  $u = \Gamma$  then
17:     $\phi \leftarrow \phi \cup p \cup u_{\text{old}}$  ▷ Add the sampled path  $p$  and the root  $u_{\text{old}}$  to  $\phi$ 
18:  else
19:     $\phi \leftarrow \phi \cup p$ 
20:  end if
21: end while
22: Output  $\phi$ 

```

This is also the slowest scenario: in this case, $\theta_C = 0$ for all cycles, and p can never contain a cycle in step 6 of Algorithm 10.1. Hence, the only way to exit the **while** loop is to reach Γ or to intersect ϕ , and we obtain the bound of Proposition 10.1 as a consequence.

Let us briefly comment on the value of the trace in Equation (10.1). We have:

$$(10.3) \quad \text{tr} \left((\mathbf{L} + \mathbf{Q})^{-1} (\mathbf{D} + \mathbf{Q}) \right) = \sum_{i \in \mathcal{V}} (\mathbf{L} + \mathbf{Q})_{i,i}^{-1} (q_i + d_i),$$

where it is known that $(\mathbf{L} + \mathbf{Q})_{i,i}^{-1}$ equals the probability that $i \in \mathbf{f} \cap \mathcal{V}$ for $\mathbf{f} \sim \mathcal{D}_{\mathcal{F}}$, and hence lies in $[0, 1]$. It follows that $\mathbf{E}_{\phi \sim \mathcal{D}_{\mathcal{M}}}(T_{\phi})$ is in $\mathcal{O} \left(\frac{|\mathcal{E}|}{q_{\min}} \right)$.

Remark 10.2. Algorithm 10.1 can be used in conjunction with Equation (9.2) to estimate $q(\widetilde{\mathbf{L}}_{\theta} + q\mathbf{I})^{-1}g$ (see Remark 4.1). In this context, we take $\mathbf{Q} = q\mathbf{D}$, and obtain the bound $\mathbf{E}_{\phi \sim \mathcal{D}_{\mathcal{M}}}(T_{\phi}) \in \mathcal{O} \left(\left(1 + \frac{1}{q}\right) |\mathcal{V}| \right)$.

10.2. Implementation-specific Complexity. The time-complexity of Algorithm 10.1 does not only depend on the number of steps taken by the random walk, but also on the cost of the cycle-detection (step 10). We propose two counter-based solutions.

One-counter detection. The first strategy consists in dynamically assigning a numerical value $c_v \in \mathbf{N}$ to each node $v \in \mathcal{V}$ encountered during the random walk (initialized to $c_v = 0$),

based on a global counter $c \in \mathbb{N}$. This counter is incremented each time a new random walk begins (instruction at line 3), and we set $c_{u'} = c$ whenever the random walk reaches u' on the instruction at line 7 (if it not already spanned by MTSF). If a cycle C is created in node u' and discarded, we reset $c_v = 0$ for all nodes v spanned by C *except* u' . A cycle can then be detected in $\mathcal{O}(1)$ time by checking the value of $c_{u'}$ at each step of the random walk: there is a cycle if $c_{u'} = c$, and no cycle if $c_{u'} < c$.

In a run of Algorithm 10.1, resetting the values c_v when cycles are discarded requires going through at most T_ϕ nodes, which results in $\mathcal{O}\left(\frac{|\mathcal{E}|}{q_{\min}}\right)$ overall expected *time complexity*.

In practice, we found it faster to use an implementation of the following strategy.

Multiple-counters detection. The c_v 's need to be reset in the one-counter cycle-detection strategy because, whenever a cycle is discarded, nodes spanned by this cycle should no longer be remembered as spanned by the path p . This expensive resetting step can be bypassed by considering *multiple* counters, in addition to the c 's (indexed by a global c). We will keep track of couples of values $(\text{id}_v, \text{val}_v) \in \mathbb{N}^2$ for each v , initialized to $(0, 0)$ and with updates based on two global counters $\text{id}, \text{val} \in \mathbb{N}^2$, and of values $\text{cap}(\text{id})$ associated to each id . id_v should be thought of as the ID of a counter at v (amongst multiple others), and val_v as its value (we only need to store the value associated to the largest id_v). These counters are reset to 0 whenever c is incremented (and a new random walk is initiated), and are otherwise updated by applying the following rules for all nodes u' reached by the random walk (if not already spanned by the MTSF).

- If no cycle is created and discarded at u' , set $\text{id}_{u'} = \text{id}$, $\text{val}_{u'} = \text{val}$, and increment val .
- If a cycle is created at node u' and discarded, store the value $\text{cap}(\text{id}) = \text{val}$ (the maximum value for the id^{th} counter), set $\text{cap}(k) = 0$ for all $k > \text{id}_{u'}$, $\text{val}_{u'} = \text{val}$, and increment id .

Using these counters, a cycle is detected at u' if $\text{val}_{u'} \leq \text{cap}(\text{id}_{u'})$.

The number id of counters used is unbounded, and we did not derive an expected time complexity for this multiple-counters strategy, but consistently obtained better performance in our measurements when using it.

Remark 10.3. The $\mathcal{O}\left(\frac{|\mathcal{E}|}{q_{\min}}\right)$ complexity can seem daunting for very small values of q_i 's, but one should remember that this is only an upper bound, not accounting for the presence of unicycles in the sample. As the $q_i \rightarrow 0$, DPP(K) approaches a distribution over spanning forests of unicycles, with a Wilson-like sampling algorithm described in [39]. Our implementation strategy can also be applied in this case and translates to the bounds on the number of steps of the random walks they derive (see also [22]).

Finally, we point out another implementation detail.

Remark 10.4. The propagation maps used in Algorithm 2.2 can be computed during the sampling process and, one actually only needs knowledge of: the root of each tree, whether or not a node i belongs to a rooted tree and, if it does, the propagation map from the root $r_\phi(i)$ to i . The actual MTSF ϕ is never used in the estimation.

11. Variance Reduction. We show that the estimators in Equations (2.12) and (2.14) are unbiased (Proposition 2.9).

1123 **Rao-Blackwell estimator.** Let us first re-define $\bar{f}$ for heterogeneous values of q . The differ-
 1124 ence resides in the aggregation function h_ϕ :

$$1125 \quad (11.1) \quad h_\phi(r, g) = \frac{\sum_{j \in c_\phi(r)} \psi_{j \xrightarrow{\phi} r} g(j)}{\sum_{r' \in c_\phi(r)} q_{r'}},$$

1126 where we recall that $c_\phi(r)$ denotes the set of nodes spanned by the tree containing r . We still
 1127 take $\bar{f}(i, \phi, g) = \psi_{r_\phi(i) \xrightarrow{\psi} i}(h_\phi(r_\phi(i), g))$, and show that:

1128 **Proposition 11.1.** $\mathbf{E}_{\phi \sim \mathcal{D}_M}(\bar{f}(i, \phi, g)) = f_*(i)$.

Proof. We will express $\bar{f}$ as a conditional expectation, the result will follow from the law of total expectation. Here, we choose a connected subset of edges $\pi \subseteq \mathcal{E}$, and condition on $\phi \cap \mathcal{E}$ containing π as one of its (maximal) components, which we denote by $\pi \sqsubseteq_{\mathcal{E}} \phi$. For i belonging to the connected component spanned by π , we have:

$$1129 \quad (11.2) \quad \mathbf{E}_{\phi \sim \mathcal{D}_M}(\tilde{f}(i, \phi, g) \mid \pi \sqsubseteq_{\mathcal{E}} \phi) = \sum_{\phi \in \mathcal{M}(\mathcal{G})} \mathbf{P}_{\mathcal{D}_M}(\phi \mid \pi \sqsubseteq_{\mathcal{E}} \phi) \tilde{f}(i, \phi, g)$$

$$1130 \quad (11.3) \quad = \sum_{\substack{\phi \in \mathcal{M}(\mathcal{G}) \\ \pi \sqsubseteq_{\mathcal{E}} \phi}} \frac{q_r \mathbf{1}_{\phi \cap c_\phi(i)}(r)}{\sum_{r' \in c_\phi(i)} q_{r'}} \tilde{f}(i, \phi, g)$$

$$1131 \quad (11.4) \quad = \bar{f}(i, \phi, g). \quad \blacksquare$$

1129 The variance of this improved estimator is derived similarly to that of $\tilde{f}$. In particular,
 1130 $\bar{f}$ is linear in g and, denoting by $\bar{S}_\phi$ the operator such that $\bar{S}_\phi g = \bar{f}(\phi, g)$, which is such that
 1131 $\bar{S}_\phi^* \mathbf{Q} \bar{S}_\phi = \mathbf{Q} \bar{S}_\phi$ and $\mathbf{E}_{\phi \sim \mathcal{D}_M}(\bar{S}_\phi) = (\mathbf{L}_\theta + \mathbf{Q})^{-1} \mathbf{Q}$, we obtain from the same argument that.

1132 **Proposition 11.2.** $\mathbf{E}_{\phi \sim \mathcal{D}_M}(\|\bar{f}(\phi, g) - f_*\|_{\mathbf{Q}}^2) = \langle g, \mathbf{Q} f_* \rangle - \|f_*\|^2$.

1133 Finally, we show the following concentration bound.

1134 **Proposition 11.3.** Let $\varepsilon, \delta \in (0, 1)$, and consider sampling m MTSFs $\{\phi_k\}_{k=1}^m$. Then, if

$$1135 \quad (11.5) \quad m \geq \frac{6}{\varepsilon^2} \log \left(\frac{|\mathcal{V}|}{\delta} \right),$$

1136 it holds for any signal $g \in \mathcal{C}^{\mathcal{V}}$ that

$$1137 \quad (11.6) \quad \mathbf{P} \left(\left\| \left(\frac{1}{m} \sum_{k=1}^m \bar{f}(\phi_k, g) \right) - f_* \right\|_2 \leq \varepsilon \|g\|_2 \right) \geq 1 - \delta.$$

1138

1139 Let us begin by stating the Matrix-Bernstein concentration bound from [80].

1140 **Theorem 11.4 (1.4 from [80]).** Consider a set of m self-adjoint matrices $\{\mathbf{X}_k\}_{k=1}^m$ in $M_n(\mathbf{C})$,
 1141 sampled independently. Assume that, almost surely and for each such matrix, both the condi-
 1142 tions

$$1143 \quad (11.7) \quad \mathbf{E}(\mathbf{X}_k) = 0$$

1144

1145 (11.8) $\|\mathbf{X}_k\| \leq R$

1146 *are satisfied, where $\|\cdot\|$ denotes the operator norm²⁹.*

1147 *Then, for all $\varepsilon \geq 0$,*

1148 (11.9)
$$\mathbf{P} \left(\left\| \sum_{k=1}^m \mathbf{X}_k \right\| \geq \varepsilon \right) \leq n e^{\frac{-\varepsilon^2}{\sigma^2 + R \frac{\varepsilon}{3}}},$$

1149 *where we define*

1150 (11.10)
$$\sigma^2 = \left\| \sum_{k=1}^m \mathbf{E}(\mathbf{X}_k^2) \right\|.$$

1151 Note that this bound is quite powerful and, in particular, that the matrices $\mathbf{X}_k$ need not be
 1152 sampled from the same distribution. Our setting is quite simple in comparison, as all the
 1153 matrices we consider *are* sampled from the same distribution.

1154 Specifically, we consider the matrices $\mathbf{X}_k = \frac{1}{m}(\bar{\mathbf{S}}_{\phi_k} - \mathbf{K})$ associated to a set of m multi-
 1155 type spanning forests $\{\phi_k\}_{k=1}^m$, with $\mathbf{K} = (\mathbf{L}_\theta + \mathbf{Q})^{-1}\mathbf{Q}$ denoting the “matrix” solution to the
 1156 connection-aware Tikhonov smoothing problem. We assume that all m multi-type spanning
 1157 forests are sampled according to $\mathcal{D}_\mathcal{M}$, and forget the subscripts in the following. Note that
 1158 we have

1159 (11.11)
$$\mathbf{E}(\mathbf{X}_k) = 0$$

1160 for all k .

1161 In order to determine R , we bound the spectrum of each $\mathbf{X}_k$:

1162 (11.12)
$$\|\mathbf{X}_k\| \leq \frac{1}{m} (\|\bar{\mathbf{S}}_{\phi_k}\| + \|\mathbf{K}\|).$$

1163 Since we have both $\|\mathbf{K}\| = \frac{q}{q+\lambda_1}$ (where λ_1 is the smallest eigenvalue of $\mathbf{L}_\theta$) and $\|\bar{\mathbf{S}}_{\phi_k}\| \leq 1$ (by,
 1164 *e.g.*, Gershgorin’s circle theorem), we can take

1165 (11.13)
$$R = \frac{1}{m} \left(1 + \frac{q}{q + \lambda_1} \right).$$

1166 In order to simplify our bounds later-on, we can notice that

1167 (11.14)
$$R \leq \frac{3}{m}.$$

²⁹Associated to the usual l_2 -norm on $\mathbf{C}^n$.

As for the value of σ^2 , we obtain from the triangle inequality that

$$(11.15) \quad \sigma^2 = \frac{1}{m} \|\mathbf{K} - \mathbf{K}^2\|$$

$$(11.16) \quad \leq \frac{1}{m} \left(\frac{q}{q + \lambda_1} + \frac{q^2}{(q + \lambda_n)^2} \right).$$

This expression can be made more interpretable by lower-bounding λ_n , and we find that:

$$(11.17) \quad \sigma^2 \leq \frac{1}{m} \left(\frac{q}{q + \lambda_1} + \frac{q^2}{(1 + q + \max_{v \in \mathcal{V}} d_v)^2} \right),$$

which is proved by leveraging the bounds on λ_n in Section 12 below.

It follows from our previous bounds on R and σ^2 that

$$(11.18) \quad \left(\sigma^2 + R \frac{\varepsilon}{3} \right) \leq \frac{1}{m} \left(\varepsilon + \frac{q}{q + \lambda_1} + \frac{q^2}{(1 + q + \max_{v \in \mathcal{V}} d_v)^2} \right).$$

Finally, $\|\sum_{k=1}^m \mathbf{X}_k\| \leq \varepsilon$ implies by definition that

$$(11.19) \quad \left\| \sum_{k=1}^m \mathbf{X}_k g \right\|_2 \leq \varepsilon \|g\|_2$$

for any signal $g \in \mathbf{C}^{\mathcal{V}}$, so that putting all the previous results together results in:

$$(11.20) \quad \mathbf{P} \left(\left\| \left(\frac{1}{m} \sum_{k=1}^m \bar{\mathbf{S}}_{\phi_k} \right) g - f_* \right\|_2 \leq \varepsilon \|g\|_2 \right) \geq 1 - \delta$$

whenever we have

$$(11.21) \quad m \geq 2 \frac{\varepsilon + \frac{q}{q + \lambda_1} + \frac{q^2}{(1 + q + \max_{v \in \mathcal{V}} d_v)^2}}{\varepsilon^2} \log \left(\frac{|\mathcal{V}|}{\delta} \right).$$

The bound from Proposition 11.3 follows when taking $\varepsilon < 1$.

Gradient step as control variates. We show that $\mathbf{E}_{\phi \sim \mathcal{D}_{\mathcal{M}}}(\hat{f}(\phi, g)) = f_*$. First recall that $\mathbf{E}_{\mathcal{D}_{\mathcal{M}}}(\bar{f}) = f_*$ (and $f_* = q(\mathbf{L}_{\theta} + q\mathbf{I})^{-1}g$) so that, by linearity of the expectation:

$$(11.22) \quad \mathbf{E}_{\phi \sim \mathcal{D}_{\mathcal{M}}}(\hat{f}(\phi, g)) = f_* - \alpha \mathbf{P}(g - g) = f_*$$

where, for heterogeneous q_i 's, we set $\mathbf{P} = (\mathbf{Q}^{-1}\mathbf{D} + \mathbf{I})^{-1}$.

This shows that $\hat{f}$ is unbiased. Let us now discuss the choice of α . We plot in Figure 11.1 the mean errors $\|\bar{f} - f_*\|$ and $\|\hat{f} - f_*\|$ over 5 trials, for different values of α . We always take $m = 20$ MTSFs, and set $q = 10^{-2} \times \bar{d}$. We use the random graph models from Section 3.

The extent of the error reduction depends on the graph, but the optimal choice of α seems to always be close to 1. This behavior can be observed across order-of-magnitude variations of q (not shown), and setting $\alpha = 1$ resulted in meaningful variance reduction for our other experiments (e.g. Section 3). This contrasts with the (similar) variance reduction technique proposed in [63], developed for connection-free graphs, for which a good choice of step-size was less straightforward.

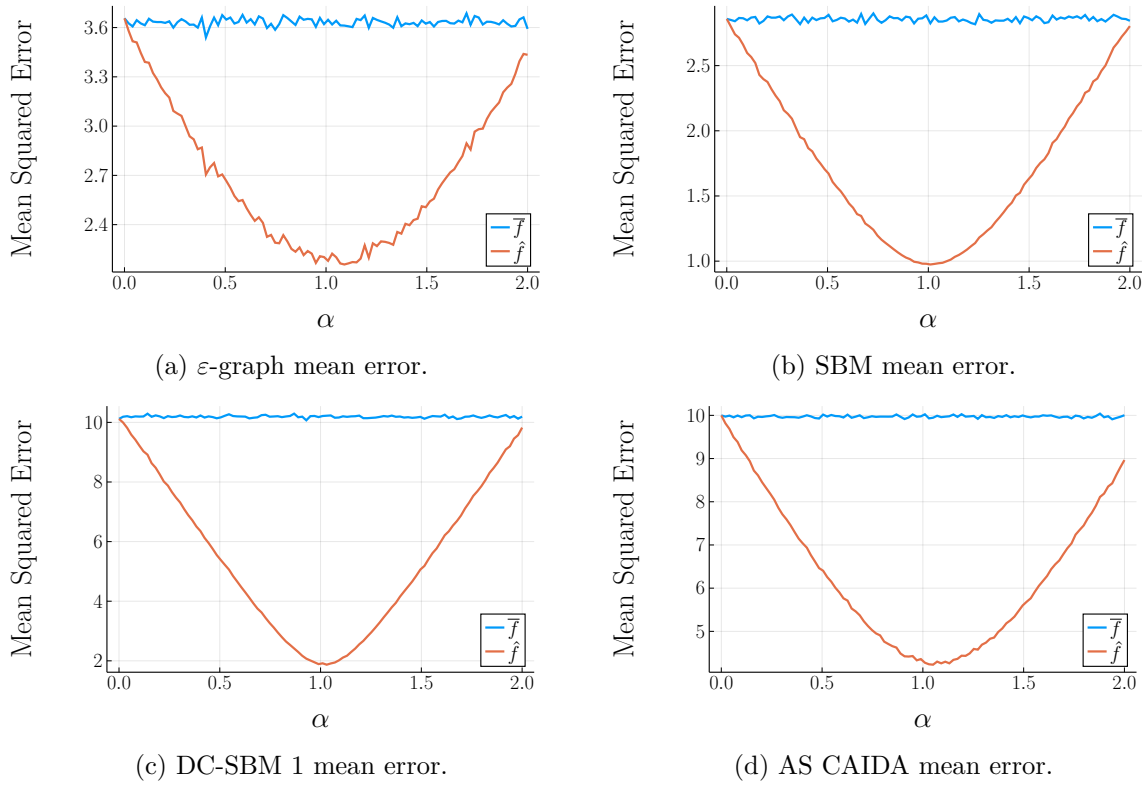


Figure 11.1: Mean error as a function of α .

12. Bounds on λ_n . We show that the largest eigenvalue λ_n of L_θ is such that

$$(12.1) \quad 1 + \max_{v \in \mathcal{V}} d_v \leq \lambda_n \leq 2 \max_{v \in \mathcal{V}} d_v,$$

which is nothing but a straightforward generalization of the same bound for the largest eigenvalue of the combinatorial Laplacian.

The right-hand-side inequality is immediately obtained from Gershgorin circle's theorem. To obtain the left-hand-side inequality, let $v_{\max} \in \mathcal{V}$ denote one vertex of $\mathcal{G}$ with maximum degree $d_{v_{\max}} = \max_{v \in \mathcal{V}} d_v$, and consider the signal $f \in \mathbb{C}^{\mathcal{V}}$ defined by

$$(12.2) \quad f(i) = \begin{cases} d_{v_{\max}} & \text{if } i = v_{\max} \\ -e^{i\theta_{v,i}} & \text{if } i \text{ is adjacent to } v_{\max} \\ 0 & \text{otherwise.} \end{cases}$$

Clearly, $\frac{\langle f, L_\theta f \rangle}{\langle f, f \rangle} = 1 + \max_{v \in \mathcal{V}} d_v$, from which the lower-bound on λ_n ensues.

13. Supporting Arguments for Tikhonov Smoothing. The goal of this Section is to argue that, when the connection is sufficiently consistent, eigenvectors of L and L_θ can be

used similarly in graph-signal-processing applications, such as the smoothing experiment in Section 3.

Coherent connection. Let us first consider a perfectly coherent connection, such that $\theta_{i,j} = \omega_j - \omega_i$ for all edges $\{i, j\}$. Consider the eigendecomposition $\mathbf{L} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^*$ of $\mathbf{L}$, and denote respectively by x the vector with entries $x_i = e^{i\omega_i}$, and $\mathbf{D}_x$ the diagonal matrix with $(\mathbf{D}_x)_{i,i} = x_i$. In this situation, we have

$$(13.1) \quad \mathbf{L}_\theta = \mathbf{D}_x \mathbf{L} \mathbf{D}_x^* = (\mathbf{D}_x \mathbf{U}) \mathbf{\Lambda} (\mathbf{D}_x \mathbf{U})^*,$$

and the eigenvectors v_i of $\mathbf{L}_\theta$ are given by $\mathbf{D}_x^* u_i$, with u_i the eigenvectors of $\mathbf{L}$.

Noisy connection. Equation (13.1) no longer holds for incoherent connections. We provide illustrations of the resulting eigenvectors of $\mathbf{L}_\theta$ for connections of the form:

$$(13.2) \quad \theta_{i,j} = \omega_j - \omega_i + \eta \varepsilon_{i,j},$$

for different values of η , and $\varepsilon_{i,j}$ uniformly distributed in $[-1, 1]$ (like in Section 3). Specifically, we represent u_i , v_i , and $\mathbf{D}_x^* v_i$ (which should approximate u_i). We also measure the difference between eigenvectors u_i and v_i as a function of η using the normalized error

$$(13.3) \quad \min_{r \in U(\mathbf{C})} \frac{\|u_i - r(\mathbf{D}_x^* v_i)\|_2}{n},$$

and use the following graphs:

- An ε -graph built from $n = 100$ points uniformly sampled in $[0, 1]^3$, with $\varepsilon = 0.3$ (Figure 13.1).
- A SBM with two communities on size 50 ($n = 100$), with $c_{1,1} = c_{2,2} = 19$ and $c_{1,2} = 1$ (Figure 13.2).

We only use a single realization of these graphs, equipped with a single realization of the connection. The normalized errors are computed across a linear range of values of η in $[0, 1]$. For each graph, we plot the real and imaginary parts of the entries of the corresponding eigenvectors, for $\eta_1 \simeq 0.11$ and $\eta_2 \simeq 0.75$. The corresponding errors are also highlighted, along with the value $\eta_0 = \frac{\pi}{2n}$ (for which we know that the connection always satisfies the weak-inconsistency Condition 2.6). Recall that the eigenvectors of $\mathbf{L}_\theta$ are only defined up to a global rotation, and this is apparent in our illustrations.

Finally, we provide similar illustrations for a cyclic graph on size $n = 100$, in a more controlled setup: the values of $\varepsilon_{i,j}$ are no longer randomly sampled, and we instead set $\varepsilon_e = 1$ for all e along a *coherent* orientation. This way, η_0 corresponds exactly to the weak-inconsistency threshold in Condition 2.6. We plot the corresponding normalized errors in Figure 13.3.

Discussion. We observe in Figure 13.1 that $\mathbf{D}_x^* v_i$ provides a good approximation of u_i for low levels of noise ($\eta = \eta_1$), but this is no longer the case at higher noise-levels ($\eta = \eta_2$): $\mathbf{D}_x^* v_2$ somewhat recovers the shape of u_2 , but $\mathbf{D}_x^* v_3$ appears completely unrelated to u_3 . Results for the SBM (Figure 13.2) suggest that major structural properties of the graph (here, the community structure) are still captured even for high noise: for both $\eta = \eta_1$ and $\eta = \eta_2$, $\mathbf{D}_x^* v_2$ clearly partition the graph into the same communities as u_i .

These behaviors are reflected in the normalized-error plots (Figure 13.1), with greater error

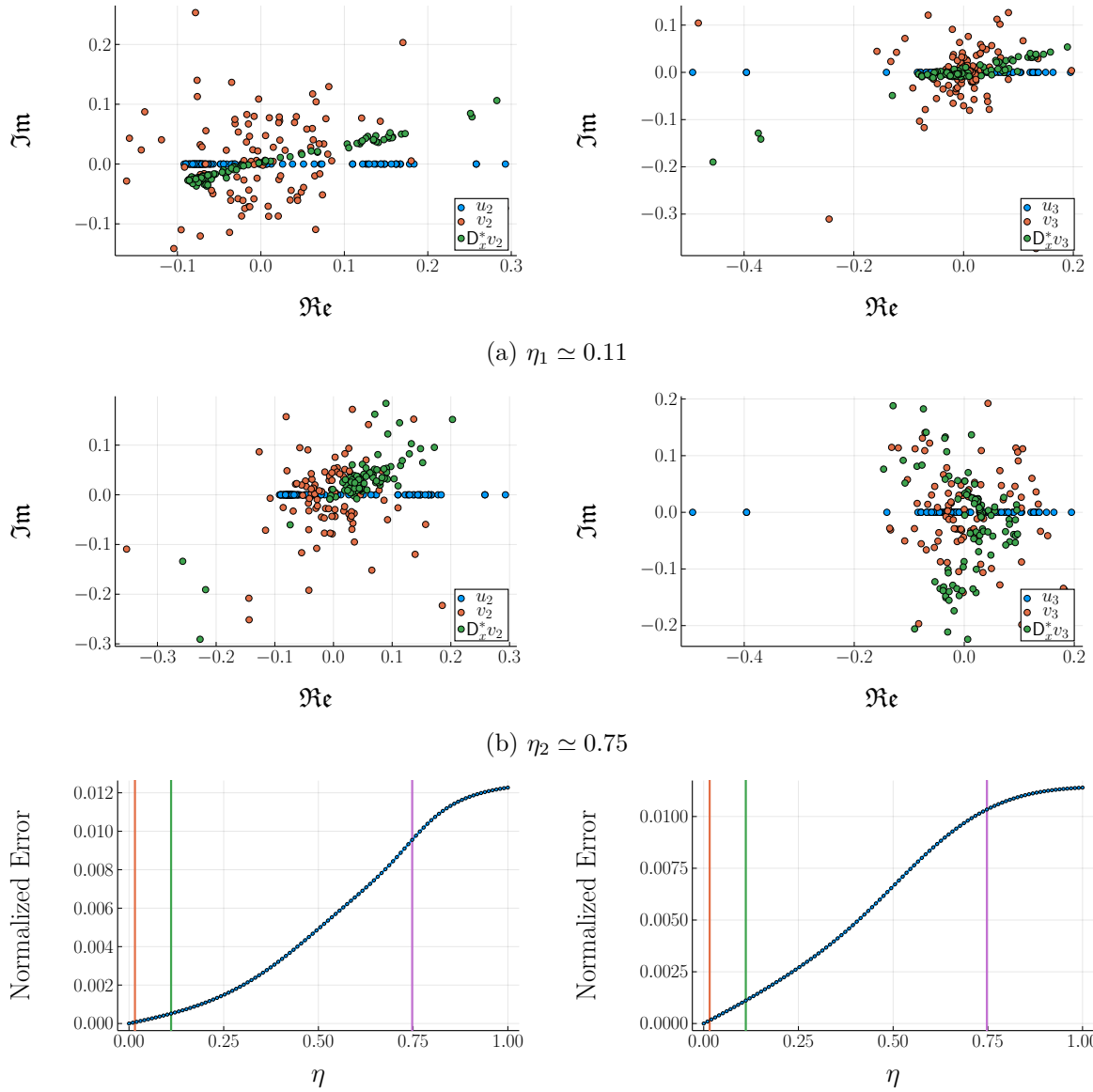


Figure 13.1: Results for the ε -graph. Left: second eigenvector. Right: third eigenvector.

1243 associated to higher values of η , and smooth decay. Results on the cycle-graph (Figure 13.3)
 1244 are very different, with three different behaviors (similar for both eigenvectors), occurring at
 1245 sharp thresholds: perfect correspondence with u_i at $\eta = 0$, two small, low-error, *plateau* (one
 1246 of them occurring as soon as $\eta > 0$, and containing $\eta = \eta_0$), and a higher, essentially constant,
 1247 error otherwise. For the SBM (not shown), the error decayed linearly with η .

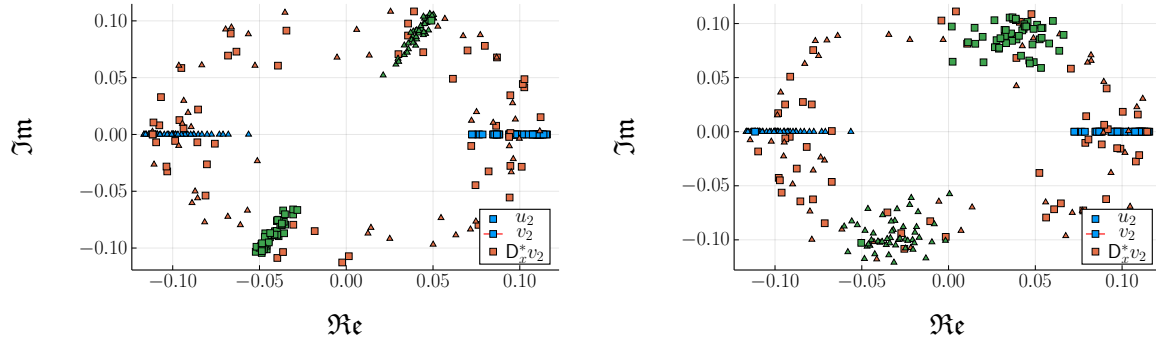


Figure 13.2: Second eigenvector for the SBM-graph. The two communities are depicted as triangles and squares respectively. Left: $\eta_1 \simeq 0.11$. Right: $\eta_2 \simeq 0.75$.

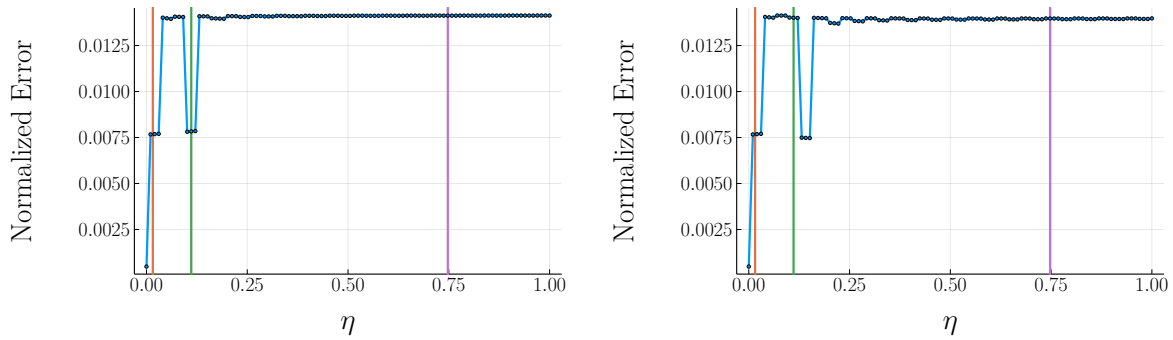


Figure 13.3: Normalized errors for the cycle-graph. Left: second eigenvector. Right: third eigenvector. Vertical lines represent η_0 (orange), η_1 (green) and η_2 respectively (purple).

These illustrations provide weak evidence that eigenvectors of $\mathbf{L}$ and $\mathbf{L}_\theta$ behave similarly in low-noise regimes. In this spirit, simple instances (*e.g.* $\omega_v = 0$ for all v but non-trivial noise) may be amenable to perturbative analyses, with $\mathbf{L}_\theta$ understood as an analytic perturbation of $\mathbf{L}$ (this is similar to the setting mentioned in Remark 1.1). The situation is much more nuanced for strongly-incoherent connections, and likely requires more involved mathematical descriptions.

14. Power iteration: $\mathbf{A}_\theta$ v.s. $q(\mathbf{L}_\theta + q\mathbf{I})^{-1}$. We compare in the following the performance of our synchronization estimators (Section 4) with that of the power iteration applied to the matrix $\mathbf{A}_\theta$ (as proposed in [75]).

Setup. We perform the experiment on three different graph/connection models, where the connection is generated according to the model of Equation (3.1) for different incoherence levels.

- With low-incoherence connections (weak-inconsistency condition satisfied), with $\eta = \frac{\pi}{2n}$, on two different random-graph models.
 - On the ε -graph model of Section 3.2.
 - On the DC-SBM 1 model of Section 3.2.

Note that, for these low-incoherence models, the spectral gaps of the matrices L_θ and A_θ should be similar to those of the matrices L and A (which we analyzed in Remark 4.1). Let us stress that low-incoherence connections of this type *are* encountered in practice, in problems such as ranking [16, 83] for instance.

- In higher-incoherence connections (no guarantee that the weak-inconsistency condition is satisfied), with $\eta = \frac{\pi}{100}$, on the same ε -graph model.

For each graph we estimate the solution $x \in U(\mathbf{C})^\mathcal{V}$ to the angular-synchronization problem by performing k power-method iterations, for $k \in \{1, 2, 4, 7, 13, 24, 45, 85, 160, 300\}$. Each power-method iteration consists in either a multiplication by A_θ , or in an approximate smoothing by $\hat{f}$ with $m = 5$ MTSFs (resp., by $m = 5$ iterations of diagonally-preconditioned CG). We set $q = 10^{-4} \times \bar{d}$, and measure for each k the resulting synchronization error (Equation (4.8)) and associated runtime. Each runtime measurement is averaged over 100 runs. The final results are averaged over 10 uniformly-selected initializations f_0 of the power method, themselves averaged over 5 graph-realizations for each model.

The results are depicted in Figure 14.1.

Comments. There are two main observations.

1. For low-inconsistence connections, the predonimant parameter is the expansivity of the graph. In particular, the spectral gap of the matrix $q(L_\theta + qI)^{-1}$ is much larger than that of A_θ for the ε -graph, and we obtain a significant gain in performance with both the $\hat{f}$ -based and CG-based iterations (for $\hat{f}$, the synchronization error is better in 2 iterations than in 300 iterations for A_θ). The situation is more nuanced for the DC-SBM 1 graph: in that case, the spectral gap of A_θ is quite small (due to the clear community structure exhibited for this graph model³⁰), so that we could expect a similar speed-up, but not actually small enough for the power-method based on A_θ to converge very slowly (as we observe in Figure 14.1)³¹.
2. For higher-incoherence connections, the bottom eigenvalue λ_1 of L_θ (resp. the top eigenvalue α_1 of A_θ) gets larger (resp. smaller), which in this example appears to slow-down (resp. speed-up) the convergence of the power-method based on the matrix $q(L_\theta + qI)^{-1}$ (resp. A_θ). This translates to a smaller improvement in the performance of our method against the power-method based on A_θ . Nevertheless, it is hard to derive a general conclusion: the eigenvalues λ_2 and α_2 are also affected by the incoherence of the connection, and the convergence rates of the power-methods depend on the *rates at which the ratios $\frac{\lambda_2+q}{\lambda_1+q}$ and $\frac{\alpha_1}{\alpha_2}$ evolve* (when the incoherence is low, this problem should be amenable to a perturbative analysis, as sketched in Section 13).

Let us note that the choice of q (which has no bearing on the end-result of the iterations, but does impact the runtime of the inverse power methods) is still subject to the trade-off discussed in Section 4: q should be small enough to allow for a fast convergence of the power iteration, but large enough so that each iteration can be performed efficiently. Here, we purposefully choose a small value of q to showcase the gain in performance of our methods

³⁰In particular, the parameters of this model ensure that we are well-above the theoretical community-reconstruction threshold for DC-SBMs (see, *e.g.* [30]).

³¹For the DC-SBM 2 model, the community structure is even more clear-cut, and we do observe a similar speed-up.

against the power-iteration based on A_θ , but do not tailor this choice to the exact graph nor to the incoherence level.

15. Illustration on MNIST images: an average digit. We propose a variant of the illustration from Section 4.2 in which the incoherence of the connection is not due to the addition of (controlled) noise, but is inherent to the data itself. Our goal is to extract from a set of similar images an “average representative” of this dataset. Here, we consider a set of 5000 “1”’s in the MNIST dataset [46] (a dataset of 28×28 handwritten digits), that all represent similar images but, due to different calligraphic styles, present some discrepancies: in particular, the images are not aligned properly, and two very similar digits may differ by a simple rotation.

For each pair of images (I, J) in the dataset, we compute a rotation $r_{I,J}$ registering the two images (based on image moments), extract for each image I its set of $s = 35$ nearest neighbors (with respect to the distance $\|I - r_{J,I}(J)\|_2$ in $\mathbf{R}^{28 \times 28}$), and build the corresponding nearest-neighbor graph and connection.

We then compute a global alignment via angular synchronization, using respectively the exact eigenvector of L_θ and a MTSF-based estimate (obtained from $\hat{f}$, with $k = 10$ power-method iterations, $m = 5$ MTSFs sampled at each step, and $q = 10^{-4} \times \bar{d}$), and average the 5000 aligned images. For comparison, we also include the average obtained without the rotations.

The results are depicted in Figure 15.1.

16. Towards Extensions: an Alternative Construction. We exhibit how the loop-erasure procedure in Algorithm 2.3 can be understood as stemming from the Feynman-Kac formula in Proposition 2.2. The aim is twofold: first, to better understand the relation between the Feynman-Kac formula in Proposition 2.2 and the MTSF-based estimator of Theorem 2.4 and, second, to propose a possible proof outline for generalizations of our work.

Loop Erasure from Feynman-Kac. Let us begin by defining an equivalence relation on paths in P_i^Γ (recall that P_i^Γ denotes the set of paths from i to Γ in the extended graph $\mathcal{G}^\Gamma$): two paths $p, q \in P_i^\Gamma$ are equivalent, which we denote by $p \simeq q$, if the only difference between the two is the orientation of (at most) *one* cycle; that is, $p \simeq q$ if $p = aCb$ and $q = aC^*b$, with C a (possibly empty) cycle in $\mathcal{G}$. For convenience, we denote by $p(C)$ and $p(C^*)$ the two possible representatives of each of the induced equivalence classes, and by $\overline{p(C)}$ the equivalence class itself.

For any node $i \in \mathcal{V}$, we can write the expected value obtained at i by performing parallel transport, from which we obtain:

$$\begin{aligned}
 (16.1) \quad \sum_{p \in P_i^\Gamma} \mathbf{P}_{\nu_i^\Gamma}(p) \psi_{p_\Gamma^*} &= \sum_{\overline{p(C)} \in P_i^\Gamma / \simeq} \left(\mathbf{P}_{\nu_i^\Gamma}(p(C)) \psi_{p(C)_\Gamma^*} + \mathbf{P}_{\nu_i^\Gamma}(p(C^*)) \psi_{p(C^*)_\Gamma^*} \right) \\
 (16.2) \quad &= \sum_{\overline{p(C)} \in P_i^\Gamma / \simeq} \mathbf{P}_{\nu_i^\Gamma}(\overline{p(C)}) (\psi_{a^*} \circ (\psi_C + \psi_{C^*}) \circ \psi_{b^*}) \\
 (16.3) \quad &= \sum_{\overline{p(C)} \in P_i^\Gamma / \simeq} 2\mathbf{P}_{\nu_i^\Gamma}(\overline{p(C)}) \left(\left(\psi_{a^*} \circ \left(\frac{\psi_C + \psi_{C^*}}{2} \right) \right) \circ \psi_{b^*} \right),
 \end{aligned}$$

where $p(C) = aCb$, and we denote by $\mathbf{P}_{\nu_i^\Gamma}(\overline{p(C)}) = \mathbf{P}_{\nu_i^\Gamma}(p(C))$ in Equation 16.2 the probability of sampling either of the two representatives $p(C)$ or $p(C^*)$ (note that $\mathbf{P}_{\nu_i^\Gamma}(p_C) = \mathbf{P}_{\nu_i^\Gamma}(p_{C^*})$). Let us re-phrase this observation in a more qualitative language: whereas each estimation in the Feynman-Kac estimator proceeds by parallel transport along a path $p(C)$, with the transport application *depending on the orientation of the cycle C* , we observe from Equation (16.3) that we can instead perform an *orientation-agnostic estimation* resulting in the same expectation, by applying the map $\frac{\psi_C + \psi_{C^*}}{2}$ instead of the connection map ψ_C ; the factor 2 in Equation (16.3) in turn accounts for the possibility of sampling either $p(C)$ or $p(C^*)$. Under the weak-inconsistency condition, this allows to probabilistically interpret the maps $\frac{\psi_C + \psi_{C^*}}{2}$ in Equation (16.3), which act as multiplications by the factors $\cos(\theta_C)$:

$$(16.4) \quad \left(\frac{\psi_C + \psi_{C^*}}{2} \right) (z) = \cos(\theta_C) \cdot z.$$

We can thus understand Equation (16.3) as a variant of the Feynman-Kac estimator involving a Bernoulli trial (with success probability $\cos(\theta_C)$), going as follows. Suppose that the path p contains a unique cycle C , and takes the form $p = aCb(j, \Gamma)$, then:

1. if the trial succeeds, set $\psi_{(ab)^*}(g(j))$ as the estimation on node i .
2. If the trial fails, set 0 as the estimation.

Let us stress that, in expectation, this procedure still results in the smoothed signal f_* .

This argument extends to any number k of cycles, and translates to a significant reduction of variance for the Feynman-Kac estimator³²: any path p with k cycles, sampled according to ν_i^Γ , belongs to an equivalence class of 2^k paths, for each of which the Feynman-Kac estimator results in a different value at node i , while the modified estimator assigns the same value to all 2^k paths.

One can go a step further, and get rid of the weak-inconsistency condition. Indeed, for any cycle C such that $\cos(\theta_C) < 0$, one can perform a Bernoulli trial with success probability $-\cos(\theta_C) = (-1) \times \cos(\theta_C)$, and multiply the resulting estimate -1 each time such a cycle is constructed in the path p , thereby *extending the estimation to any $U(\mathbf{C})$ -connection*.

The crucial observation follows, as one can identify:

- the modified Feynman-Kac estimator, and in particular the Bernoulli trial for each cycle;
- the cycle-acceptance trial of Algorithm 10.1.

More specifically, consider the first node i in the queue of the MTSF-sampling algorithm, at which the first loop-erased random walk starts: this walk constructs a path p and, at each encountered cycle C in $\mathcal{G}$, may be interrupted due to cycle-acceptance, with probability $1 - \cos(\theta_C)$, in which case the ensuing estimation at node i is $\tilde{f}(i) = 0$.

Similarly, consider the modified Feynman-Kac estimator, which performs successive Bernoulli trials with success probability $\cos(\theta_C)$, and notice that the result of the estimation at node i is decided *as soon as a trial fails*, in which case it is going to be 0. Effectively, the resulting path

³²Note that this requires to be careful in the definition of the successive cycles, as one cycle may be contained within another. Here, a convenient choice of definition is to define the set of cycles according to the following procedure: whenever a cycle is observed in the path p constructed in the random walk, add it to the set of cycles and remove it from p .

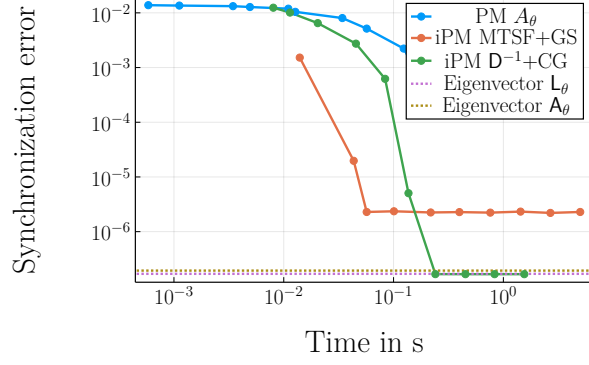
p upon which the parallel transport will be applied describes a part of a multi-type spanning forest: the branch on a unicycle, and the associated cycle.

In both cases, if no cycle is accepted (resp. no Bernoulli trial fails), the walk ends upon reaching node Γ , and the estimation at node i is described by the parallel transport from Γ to i along the path $p \subseteq \mathcal{E}$.

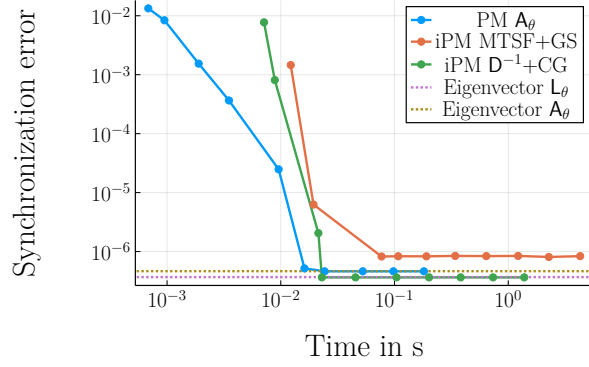
The question is then the following. Can such a construction be extended to *all* the nodes in $\mathcal{G}$, so as to recover the (unbiasedness of the) MTSF-based estimator $\tilde{f}$? This question is important for a few reasons.

- First and foremost, this construction could both allow to extend MTSF-based estimators to non-weakly-inconsistent connections, and provide a much more intuitive construction of multi-type spanning forests.
- Second, it is likely to be more flexible: instead of interpreting the factors $\frac{\psi_C + \psi_{C^*}}{2}$ probabilistically, one could, *e.g.*, systematically apply the map $\frac{\psi_C + \psi_{C^*}}{2}$, which would result in connection-aware MTSF-based estimators *with no unicycles*.

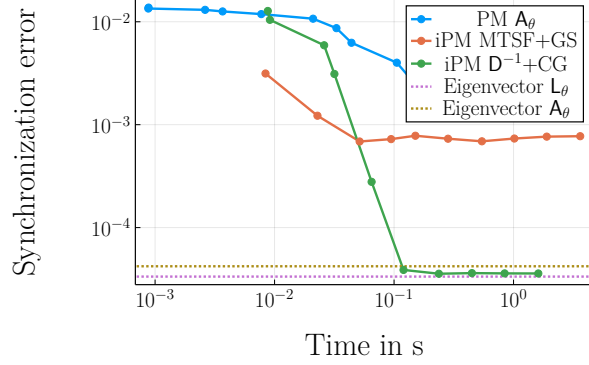
Remark 16.1. *A similar cycle-equivalence argument has also been used to prove the determinantal-ity of the spanning-forest-of-unicycles measure of [25, 41] in [38] (in the weakly-inconsistent setting) and, more recently, to derive the exact expected number of steps for Algorithm 10.1 algorithm in [23].*



(a) ε -graph, $\eta = \frac{\pi}{2n}$.

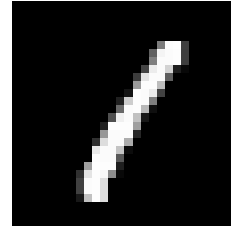
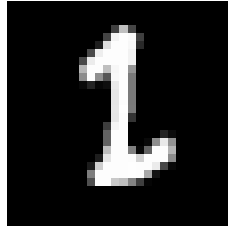
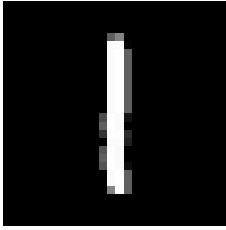


(b) DC-SBM 1, $\eta = \frac{\pi}{2n}$.

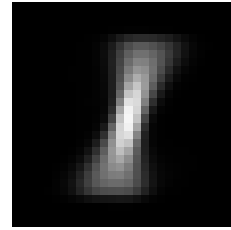
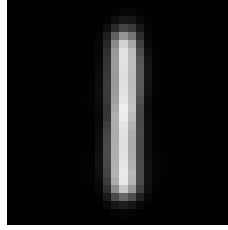
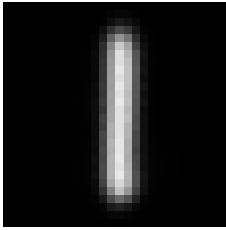


(c) ε -graph, $\eta = \frac{\pi}{100}$.

Figure 14.1: Runtime-precision comparisons for the power iteration based on A_θ [75] v.s. the approximate (inverse) power methods from Section 4. The results obtained from the exact eigenvectors of L_θ and A_θ are represented as horizontal lines.



(a) Some “1”’s from the MNIST dataset.



(b) Exact eigenvector.

(c) $\bar{f}$ -based estimation.

(d) No rotation.

Figure 15.1: Top: some “1”’s from the MNIST dataset. Bottom: average “1” in the dataset, as obtained by three different methods (left: exact synchronization; center: MTSF-based synchronization; right: no synchronization). The results of the synchronization are only given up to a global rotation, which is apparent in the recovered images.